Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

WSQ CompTIA Certified Cloud+ Training

TGS Ref No: TGS-2024049214

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.1

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	1 Jul 2026	First release. 36 hands-on labs mapped to CV0-004 domains 1 –6.	Dr. Alfred Ang
2.0	19 Aug 2026	Major revision. Deck expanded with full per-domain teaching content covering every CV0-004 exam sub-objective (31 blueprint topics, 500+ visual slides); labs restructured into one folder per lab; free web-tool kit (IP Calculator, PCAP Analyzer, Regex Generator, Cybersecurity Simulator) integrated into the labs.	Dr. Alfred Ang
2.1	19 Aug 2026	Lab platforms split by workload: Kubernetes labs move to the Killercoda Kubernetes playground, container labs may run on Docker Desktop, Linux labs stay on the Ubuntu playground. Each lab, slide and Learner Guide section now names its platform. Labs ship 41 ready-made asset files (Dockerfiles, Compose files, Kubernetes manifests, Terraform, Ansible playbooks, configs and scripts) so learners can download instead of retyping.	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

How to Use This Guide

Welcome to the **WSQ CompTIA Certified Cloud+ Training** (CV0-004) hands-on Learner Guide, conducted by Tertiary Infotech Academy Pte Ltd (UEN 201200696W).

This guide contains **36 step-by-step labs** mapped to the six CompTIA Cloud+ CV0-004 exam domains. Every lab runs free in your browser — no cloud account and no credit card required.

Lab Platforms

Each lab names the platform it runs on at the top of the lab. There are three:

Platform	Use it for	Link
Killercoda Ubuntu Playground	The Linux labs — a disposable root Ubuntu VM in the browser. This is the default for most labs.	https://killercoda.com/playgrounds/scenario/ubuntu
Killercoda Kubernetes Playground	The Kubernetes labs — a real cluster with `kubectl` already configured. The Ubuntu playground has no cluster.	https://killercoda.com/playgrounds/scenario/kubernetes
Docker Desktop	The container labs, run locally on Windows, macOS or Linux. Optional — the same labs also work on the Ubuntu playground.	https://www.docker.com/products/docker-desktop/

Which lab uses which platform

- **Killercoda Kubernetes Playground:** Lab 32
- **Docker Desktop (or the Ubuntu playground):** Lab 6, 12, 36
- **Killercoda Ubuntu Playground:** the remaining 32 labs

> *On Docker Desktop the engine is already running, so skip any `apt install docker.io` / `systemctl start docker` step. Everything else is identical.*

Before you start:

- Open the platform named at the top of the lab you are about to do.
- Work through the labs in order; each one builds on the mindset of the last.
- **Reset the playground** between labs that change firewall, container or kernel state.
- Copy the commands exactly as shown and read the short explanation under each step.
- Labs that build config files, manifests or scripts ship them as **ready-made files** in the lab folder — download them instead of typing, if you prefer.
- The full lab source is also on GitHub: https://github.com/tertiarycourses/TGS-2024049214-CompTIA-Certified-Cloud-Training

What You Will Achieve

- Explain cloud architecture, service models (IaaS/PaaS/SaaS/FaaS) and the shared responsibility model, and design highly-available, well-networked cloud solutions.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Deploy cloud workloads using containers, virtualization, Infrastructure as Code and modern release strategies (blue-green, canary, rolling).
- Operate cloud systems with observability, logging, tracing, auto-scaling, backup/recovery and lifecycle management.
- Secure a cloud environment with vulnerability management, hardening, IAM/RBAC, secrets management, WAF and runtime threat detection.
- Apply DevOps fundamentals — source control, CI/CD, APIs and container orchestration.
- Troubleshoot common cloud deployment, network, security and container issues methodically.

Exam at a Glance

Item	Detail
Exam code	CV0-004
Questions	Maximum of 90
Question types	Multiple-choice and performance-based
Length	90 minutes
Passing score	750 (on a scale of 100–900)
Recommended experience	2–3 years as a systems administrator / cloud engineer; CompTIA Network+ and Server+ or equivalent knowledge

Domain	Exam Weight	Labs
1. Cloud Architecture	23%	1, 2, 3, 4, 5, 6, 7, 8
2. Deployment	19%	9, 10, 11, 12, 13, 14, 15
3. Operations	17%	16, 17, 18, 19, 20, 21
4. Security	19%	22, 23, 24, 25, 26, 27, 28
5. DevOps Fundamentals	10%	29, 30, 31, 32
6. Troubleshooting	12%	33, 34, 35, 36

Domain 1 — Cloud Architecture (23%)

Cloud concepts, service and deployment models, the shared responsibility model, high availability, networking (VPC), storage tiers, virtualization, databases and cloud-native / microservices design.

What this domain covers in the labs:

- **Lab 1 — Cloud Service Models & Shared Responsibility:** Run a local stand-in for each cloud service model and articulate who is responsible for what under the shared responsibility model.
- **Lab 2 — High Availability with HAProxy & Keepalived:** Build a health-checked load balancer across two AZ backends and add VRRP failover so a virtual IP survives a load-balancer failure.
- **Lab 3 — Cloud Networking with VPC Namespaces:** Construct a full VPC — two subnets, a router, a security-group rule and a NAT gateway — entirely from Linux network namespaces.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Lab 4 — Storage Tiers (Block, Object, File):** Create one block, one object and one file store locally, then benchmark IOPS to explain why storage tiers (and their prices) differ.
- **Lab 5 — Microservices & Service Discovery:** Deploy three loosely-coupled microservices behind Consul and demonstrate name-based service discovery, fan-out and resilience under a single failure.
- **Lab 6 — Containerization with Docker:** Practise every core containerization sub-objective — build/run/expose, ephemeral vs persistent storage, public/private registries and a Compose preview.
- **Lab 7 — Virtualization with QEMU/KVM:** Create and clone a QEMU VM to understand core virtualization concepts and contrast VMs with containers.
- **Lab 8 — Relational vs Non-Relational Databases:** Run and compare a self-managed relational DB against a document DB to learn when to choose each and how managed services shift responsibility.

Lab 1 — Cloud Service Models & Shared Responsibility

Goal: Run a local stand-in for each cloud service model and articulate who is responsible for what under the shared responsibility model.

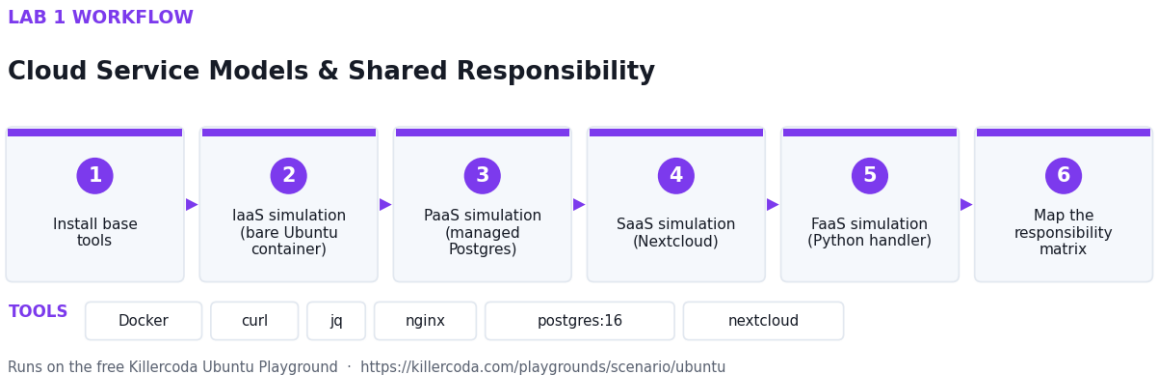

Figure 1 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Docker, curl, jq, nginx, postgres:16, nextcloud

In this lab you will compare IaaS, PaaS, SaaS, and FaaS by running the equivalent of each model **locally** on the Killercoda Ubuntu Playground. You will provision a raw VM-style workload (IaaS), deploy a managed Postgres-like service (PaaS), use a SaaS-style web app, and run a Function-as-a-Service handler. By the end you will be able to explain who is responsible for what under the **shared responsibility model**.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Update and install base tools

```
apt update && apt install -y docker.io curl jq python3-pip
systemctl start docker
```

Docker will let us simulate cloud-managed services as containers. `curl` and `jq` are used to call APIs.

Step 2 — IaaS simulation (you manage everything above the hypervisor)

Run a bare Ubuntu container — you patch it, you secure it, you install the runtime. This mirrors AWS EC2 / Azure VM / GCP Compute Engine.

```
docker run -dit --name iaas-vm ubuntu:22.04 bash
docker exec iaas-vm apt update
docker exec iaas-vm apt install -y nginx
docker exec iaas-vm service nginx start
```

You installed the OS package, configured the service, and started it. Under IaaS the **customer owns OS, runtime, app, data** and the **provider owns hardware, hypervisor, network**.

Step 3 — PaaS simulation (provider manages the runtime)

Run a managed-style Postgres. You only supply the database name and password — no OS patching.

```
docker run -d --name paas-db -e POSTGRES_PASSWORD=cloud -p 5432:5432
postgres:16
sleep 5
docker exec paas-db psql -U postgres -c "CREATE DATABASE app;"
```

You did not install Postgres, configure `pg_hba.conf`, or apply OS patches. Under PaaS the **provider owns OS + runtime**, the **customer owns app + data**.

Step 4 — SaaS simulation (you only consume)

Run a fully packaged app — Nextcloud — the way you would consume Microsoft 365 or Salesforce.

```
docker run -d --name saas-app -p 8080:80 nextcloud:latest
sleep 10
curl -sI http://localhost:8080 | head -3
```

You did not write code, manage runtime, or touch the database. Under SaaS the **provider owns everything except your data and access**.

Step 5 — FaaS simulation (event-driven function)

Install OpenFaaS-style local handler with a Python script that runs on demand — like AWS Lambda.

Ready-made file: `handler.py` — you can download it instead of typing this block.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mkdir -p /tmp/faas && cat > /tmp/faas/handler.py <<'EOF'
import sys, json
event = json.loads(sys.stdin.read())
print(json.dumps({"hello": event.get("name", "cloud")}))
EOF

echo '{"name":"Cloud+"}' | python3 /tmp/faas/handler.py
```

The function runs only when invoked, scales to zero, and bills per-invocation. The **provider owns the entire execution environment**; you only own the function code.

Step 6 — Map responsibility

Fill in the matrix from what you just ran:

Layer	IaaS	PaaS	SaaS	FaaS
Data	You	You	You	You
Application	You	You	Provider	You (code only)
Runtime	You	Provider	Provider	Provider
OS	You	Provider	Provider	Provider
Hypervisor / Hardware	Provider	Provider	Provider	Provider

Step 7 — Cleanup

```
docker rm -f iaas-vm paas-db saas-app
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker ps --filter name=iaas-vm --filter name=paas-db --filter name=saas-app
docker exec iaas-vm service nginx status
docker exec paas-db psql -U postgres -lqt | grep app
curl -sI http://localhost:8080 | head -1
echo '{"name":"Cloud+"}' | python3 /tmp/faas/handler.py
```

Expected: Run this before Step 7. `docker ps` lists all three containers (`iaas-vm`, `paas-db`, `saas-app`) as **Up**, nginx inside `iaas-vm` reports *nginx is running*, the `app` database appears in the Postgres list, Nextcloud answers with an `HTTP/1.1 200 OK` or `302 Found` status line, and the FaaS handler prints `{"hello": "Cloud+"}`.

What you learned

- Hands-on difference between IaaS, PaaS, SaaS, and FaaS.
- Who patches the OS, who owns the data, and who runs the runtime in each model.
- The shared responsibility model is **never** "the provider does it all".

Free tools used

- Docker — https://www.docker.com
- Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu
- AWS Shared Responsibility Model (reading) — https://aws.amazon.com/compliance/shared-responsibility-model/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Files in this lab

File	Purpose
`handler.py`	Step 5 FaaS handler — reads a JSON event on stdin and returns a greeting.

Lab 2 — High Availability with HAProxy & Keepalived

Goal: Build a health-checked load balancer across two AZ backends and add VRRP failover so a virtual IP survives a load-balancer failure.

Figure 2 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: HAProxy, Keepalived, Docker, nginx:alpine, curl

In this lab you will simulate a multi-AZ load-balanced deployment by running two backend web servers and an HAProxy load balancer in front of them. You will then add a second HAProxy instance with **Keepalived** so a virtual IP fails over when the primary load balancer dies — the same pattern used by every cloud provider's regional load balancer.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install HAProxy and Keepalived

```
apt update && apt install -y haproxy keepalived docker.io curl
systemctl start docker
```

Step 2 — Launch two backend "AZ" web servers

Each container represents a server in a different Availability Zone.

```
docker run -d --name az1 -p 8081:80 nginx:alpine
docker run -d --name az2 -p 8082:80 nginx:alpine
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker exec az1 sh -c 'echo "AZ-1 healthy" > /usr/share/nginx/html/index.html'
docker exec az2 sh -c 'echo "AZ-2 healthy" > /usr/share/nginx/html/index.html'

curl http://localhost:8081
curl http://localhost:8082
```

Step 3 — Configure HAProxy with health checks

Ready-made file: `haproxy.cfg` — you can download it instead of typing this block.

```
cat > /etc/haproxy/haproxy.cfg <<'EOF'
global
    daemon
    maxconn 256

defaults
    mode http
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend web
    bind *:80
    default_backend azs

backend azs
    balance roundrobin
    option httpchk GET /
    server az1 127.0.0.1:8081 check
    server az2 127.0.0.1:8082 check
EOF

systemctl restart haproxy
```

Step 4 — Verify load balancing

```
for i in 1 2 3 4; do curl -s http://localhost/; done
```

You should see responses alternate between AZ-1 and AZ-2 — round-robin across availability zones.

Step 5 — Simulate an AZ failure

```
docker stop az1
sleep 3
for i in 1 2 3; do curl -s http://localhost/; done
```

HAProxy's health check marks `az1` down within seconds and routes 100% of traffic to AZ-2. This is the equivalent of an **AZ outage** in a cloud region.

```
docker start az1
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 6 — Add Keepalived for VRRP failover

> *Ready-made file: `keepalived.conf` — you can download it instead of typing this block.*

```
cat > /etc/keepalived/keepalived.conf <<'EOF'
vrrp_instance VI_1 {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 100
    advert_int 1
    authentication { auth_type PASS; auth_pass cloud }
    virtual_ipaddress { 10.0.0.250/24 }
}
EOF

systemctl start keepalived
ip -brief addr show eth0
```

The VIP `10.0.0.250` is now floating on this node. In a real two-node setup, when MASTER dies the BACKUP picks up the VIP within ~3 seconds — this is how regional **active-passive** load balancers achieve HA.

Step 7 — Cleanup

```
systemctl stop keepalived haproxy
docker rm -f az1 az2
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker ps --filter name=az1 --filter name=az2 --format '{{.Names}}
\t{{.Status}}'
for i in 1 2 3 4; do curl -s http://localhost/; done
systemctl is-active haproxy
ip -brief addr show eth0 | grep 10.0.0.250
```

Expected: Run this before Step 7. Both `az1` and `az2` show as **Up**, the four `curl` calls through HAProxy alternate `AZ-1 healthy` / `AZ-2 healthy` (round-robin across the two simulated availability zones), `haproxy` reports `active`, and the Keepalived VIP `10.0.0.250/24` is listed on `eth0`.

What you learned

- How a cloud load balancer routes traffic across availability zones.
- How active health checks remove unhealthy instances automatically.
- How Keepalived/VRRP provides a floating IP for LB redundancy.

Free tools used

- HAProxy — https://www.haproxy.org
- Keepalived — https://www.keepalived.org
- Docker — https://www.docker.com

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Files in this lab

File	Purpose
`haproxy.cfg`	Step 3 HAProxy config — round-robin across the two simulated AZ backends with health checks.
`keepalived.conf`	Step 6 Keepalived VRRP config — floats the virtual IP 10.0.0.250.

Lab 3 — Cloud Networking with VPC Namespaces

Goal: Construct a full VPC — two subnets, a router, a security-group rule and a NAT gateway — entirely from Linux network namespaces.

Figure 3 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: iproute2 (ip netns), iptables, sysctl, ping

In this lab you will build a virtual private cloud (VPC) entirely with Linux network namespaces. You will create two subnets, a router, NAT to the internet, and a security group rule — exactly the building blocks AWS, Azure and GCP expose as managed services.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install tools

```
apt update && apt install -y iproute2 iptables iputils-ping
```

Step 2 — Create the VPC subnets

A network namespace is an isolated Linux network stack — equivalent to a subnet plus its attached ENIs.

```
ip netns add subnet-a
ip netns add subnet-b
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ip netns add router

ip netns list
```

Step 3 — Wire the subnets to the router

Use veth pairs as virtual cables.

```
ip link add a-r type veth peer name r-a
ip link add b-r type veth peer name r-b

ip link set a-r netns subnet-a
ip link set r-a netns router
ip link set b-r netns subnet-b
ip link set r-b netns router
```

Assign IPs (10.0.1.0/24 and 10.0.2.0/24):

```
ip -n subnet-a addr add 10.0.1.10/24 dev a-r
ip -n router   addr add 10.0.1.1/24  dev r-a
ip -n subnet-b addr add 10.0.2.10/24 dev b-r
ip -n router   addr add 10.0.2.1/24  dev r-b

for ns in subnet-a subnet-b router; do
  ip -n $ns link set lo up
done

ip -n subnet-a link set a-r up
ip -n subnet-b link set b-r up
ip -n router link set r-a up
ip -n router link set r-b up
```

Step 4 — Add default routes (the route table)

```
ip -n subnet-a route add default via 10.0.1.1
ip -n subnet-b route add default via 10.0.2.1
ip netns exec router sysctl -w net.ipv4.ip_forward=1
```

Test inter-subnet connectivity (peering):

```
ip netns exec subnet-a ping -c 2 10.0.2.10
```

Step 5 — Add a security group rule (deny ICMP from B to A)

```
ip netns exec router iptables -I FORWARD -s 10.0.2.0/24 -d 10.0.1.0/24 -p icmp
-j DROP
ip netns exec subnet-b ping -c 2 -W 2 10.0.1.10 || echo "Blocked by SG"
```

This is the equivalent of an AWS Security Group / Azure NSG / GCP Firewall rule.

Step 6 — Add NAT gateway to the internet

```
ip link add nat-r type veth peer name r-nat
ip link set r-nat netns router
ip addr add 192.168.99.1/24 dev nat-r
ip -n router addr add 192.168.99.2/24 dev r-nat
ip link set nat-r up
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
ip -n router link set r-nat up
ip -n router route add default via 192.168.99.1

sysctl -w net.ipv4.ip_forward=1
iptables -t nat -A POSTROUTING -s 10.0.0.0/16 -o eth0 -j MASQUERADE

ip netns exec subnet-a ping -c 2 8.8.8.8 || echo "Outbound test"
```

This mirrors a cloud **NAT Gateway** for private subnets reaching the internet.

Step 7 — Cleanup

```
ip netns del subnet-a
ip netns del subnet-b
ip netns del router
ip link del nat-r 2>/dev/null
iptables -t nat -F POSTROUTING
```

Test it

Run these checks to prove the lab worked before you move on:

```
ip netns list
ip netns exec subnet-a ip route show
ip netns exec subnet-a ping -c 2 10.0.2.10
ip netns exec router iptables -L FORWARD -n --line-numbers
ip netns exec subnet-a ping -c 2 -W 2 8.8.8.8
```

Expected: Run this before Step 7. `ip netns list` shows `subnet-a`, `subnet-b` and `router`; `subnet-a` has a `default via 10.0.1.1` route; the ping to `10.0.2.10` gets **2 packets transmitted, 2 received, 0% packet loss** across the router (VPC peering works); the FORWARD chain shows the `DROP ... icmp` rule from 10.0.2.0/24 to 10.0.1.0/24 (the security group); and the ping to `8.8.8.8` leaves through the NAT MASQUERADE rule.

What you learned

- VPCs, subnets, route tables, security groups, and NAT gateways are kernel features.
- Peering = a route + a forwarding rule.
- Security groups operate per-flow at L3/L4.

Free tools used

- iproute2 / iptables (built-in)
- TIS IP Calculator — https://alfredang.github.io/ipcalculator/

Lab 4 — Storage Tiers (Block, Object, File)

Goal: Create one block, one object and one file store locally, then benchmark IOPS to explain why storage tiers (and their prices) differ.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

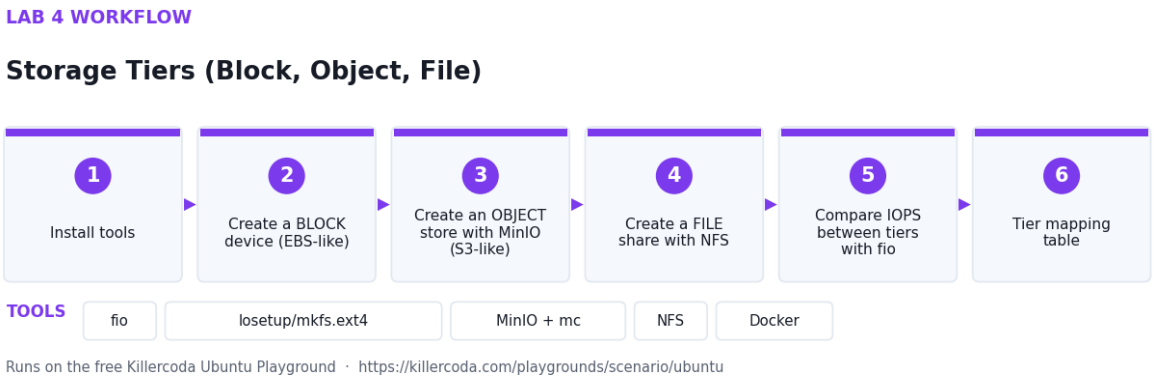

Figure 4 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: fio, losetup/mkfs.ext4, MinIO + mc, NFS, Docker

In this lab you will create one of each cloud storage type on the Killercoda VM: a **block** device with a loopback file, an **object** store with MinIO (S3-compatible), and a **file** share with NFS. You will then measure IOPS with `fio` to see why hot/warm/cold tiering exists.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install tools

```
apt update && apt install -y fio nfs-kernel-server nfs-common docker.io
systemctl start docker
```

Step 2 — Create a BLOCK device (like AWS EBS)

```
dd if=/dev/zero of=/tmp/disk.img bs=1M count=256
LOOP=$(losetup -f --show /tmp/disk.img)
mkfs.ext4 $LOOP
mkdir -p /mnt/block
mount $LOOP /mnt/block
df -h /mnt/block
```

Block storage = raw device, formatted with a filesystem, mounted to a single VM.

Step 3 — Create an OBJECT store (like AWS S3)

```
docker run -d --name minio -p 9000:9000 -p 9001:9001 \
  -e MINIO_ROOT_USER=admin -e MINIO_ROOT_PASSWORD=cloudplus \
  minio/minio server /data --console-address ":9001"

sleep 5

docker exec minio mc alias set local http://127.0.0.1:9000 admin cloudplus
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker exec minio mc mb local/hot-bucket
echo "Hello Cloud+" > /tmp/file.txt
docker cp /tmp/file.txt minio:/file.txt
docker exec minio mc cp /file.txt local/hot-bucket/
docker exec minio mc ls local/hot-bucket/
```

Object storage = HTTP API, infinite scale, no filesystem, accessed by key.

Step 4 — Create a FILE share (like AWS EFS / Azure Files)

```
mkdir -p /srv/nfs && chmod 777 /srv/nfs
echo '/srv/nfs *(rw,sync,no_subtree_check,no_root_squash)' > /etc/exports
exportfs -ra
systemctl start nfs-kernel-server

mkdir -p /mnt/file
mount -t nfs 127.0.0.1:/srv/nfs /mnt/file
echo "shared" > /mnt/file/test.txt
cat /srv/nfs/test.txt
```

File storage = POSIX filesystem, shared by many clients over NFS or SMB.

Step 5 — Compare IOPS between tiers (hot vs cold)

```
echo "--- BLOCK (hot, SSD-like) ---"
fio --name=hot --filename=/mnt/block/test --size=64M --rw=randwrite --bs=4k --
runtime=5 --time_based --group_reporting | grep IOPS

echo "--- FILE over NFS (warm) ---"
fio --name=warm --filename=/mnt/file/test --size=64M --rw=randwrite --bs=4k --
runtime=5 --time_based --group_reporting | grep IOPS
```

You will see block storage delivers significantly higher random-write IOPS than NFS — this is why **hot** tier (SSD block) costs more than **warm** (file) and far more than **cold/archive** (object).

Step 6 — Tier mapping

Tier	Cloud example	This lab
Hot (SSD block)	AWS gp3, Azure Premium SSD	`/mnt/block` (ext4)
Warm (file)	AWS EFS, Azure Files	`/mnt/file` (NFS)
Cold (object std)	AWS S3 Standard	MinIO `hot-bucket`
Archive	AWS Glacier, Azure Archive	MinIO with lifecycle

Step 7 — Cleanup

```
umount /mnt/block /mnt/file
losetup -d $LOOP
docker rm -f minio
systemctl stop nfs-kernel-server
```

Test it

Run these checks to prove the lab worked before you move on:

```
df -h /mnt/block /mnt/file
mount | grep -E 'loop|nfs'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker exec minio mc ls local/hot-bucket/
cat /srv/nfs/test.txt
exportfs -v
```

Expected: Run this before Step 7. `/mnt/block` shows a ~250M ext4 filesystem on a `/dev/loop*` device and `/mnt/file` shows the `127.0.0.1:/srv/nfs` NFS mount; `mc ls` lists `file.txt` in `hot-bucket`; `cat` prints `shared` (proving the NFS write landed in `/srv/nfs`); and `exportfs -v` shows `/srv/nfs` exported with `rw`.

What you learned

- Block, object, and file storage have different APIs and access patterns.
- IOPS varies by an order of magnitude between tiers.
- Cost models follow performance — hot is fastest and most expensive.

Free tools used

- MinIO — https://min.io
- NFS kernel server (built-in)
- fio — https://github.com/axboe/fio

Lab 5 — Microservices & Service Discovery

Goal: Deploy three loosely-coupled microservices behind Consul and demonstrate name-based service discovery, fan-out and resilience under a single failure.

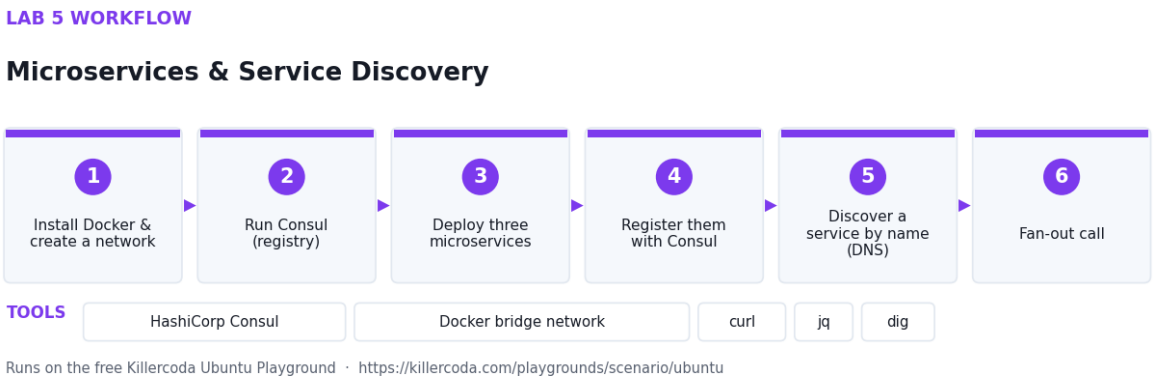

Figure 5 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: HashiCorp Consul, Docker bridge network, curl, jq, dig

In this lab you will deploy three loosely-coupled microservices behind Consul for service discovery. You will see how containers register themselves, how a client resolves a service by name (not IP), and how the fan-out pattern works when one service calls many.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Docker and create a network

```
apt update && apt install -y docker.io curl jq
systemctl start docker
docker network create cloudnet
```

Step 2 — Run Consul (service registry)

```
docker run -d --name consul --network cloudnet \
  -p 8500:8500 hashicorp/consul:latest \
  agent -dev -client=0.0.0.0
sleep 3
curl -s http://localhost:8500/v1/status/leader
```

Step 3 — Deploy three microservices

Each service is a tiny HTTP responder.

```
for n in users orders payments; do
  docker run -d --name svc-$n --network cloudnet \
    -e PORT=80 \
    nginx:alpine
  docker exec svc-$n sh -c "echo '{\"service\":\"$n\",\"ok\":true}' >
/usr/share/nginx/html/index.html"
done
```

Step 4 — Register them with Consul

```
for n in users orders payments; do
  curl -s -X PUT -d "{
    \"Name\": \"$n\",
    \"Address\": \"svc-$n\",
    \"Port\": 80,
    \"Check\": {\"HTTP\": \"http://svc-$n/\", \"Interval\": \"10s\"}
  }" http://localhost:8500/v1/agent/service/register
done

curl -s http://localhost:8500/v1/catalog/services | jq
```

Step 5 — Discover a service by name (DNS interface)

Consul exposes service names over DNS on port 8600.

```
docker run --rm --network cloudnet --dns 172.17.0.1 alpine \
  sh -c "apk add --quiet bind-tools && dig @consul -p 8600
users.service.consul"
```

Clients now resolve `users.service.consul` instead of hard-coded IPs — the **service discovery** primitive of every cloud-native platform.

Step 6 — Fan-out call

Simulate one request triggering parallel calls to all three services (event fan-out).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker run --rm --network cloudnet alpine sh -c "
  apk add --quiet curl
  for s in users orders payments; do
    (curl -s http://svc-$s/ &)
  done
  wait
"
```

Step 7 — Loosely coupled: kill one service, others survive

```
docker stop svc-orders
curl -s http://localhost:8500/v1/health/service/users | jq '.
[].Checks[].Status'
curl -s http://localhost:8500/v1/health/service/orders | jq '.
[].Checks[].Status'
```

`users` stays `passing`, `orders` flips to `critical` — but the rest of the system keeps running. That is **loose coupling**.

Step 8 — Cleanup

```
docker rm -f consul svc-users svc-orders svc-payments
docker network rm cloudnet
```

Test it

Run these checks to prove the lab worked before you move on:

```
curl -s http://localhost:8500/v1/status/leader
curl -s http://localhost:8500/v1/catalog/services | jq
curl -s http://localhost:8500/v1/health/service/users | jq '.
[].Checks[].Status'
docker ps --filter name=svc- --format '{{.Names}}\t{{.Status}}'
```

Expected: Run this before Step 8. Consul returns a leader address such as `"127.0.0.1:8300"`, the catalog lists `users`, `orders` and `payments` alongside `consul`, the `users` health check returns `"passing"`, and all three `svc-*` containers are **Up** (after Step 7, `svc-orders` is stopped and its check reads `"critical"` while `users` stays `"passing"` — that is the loose coupling).

What you learned

- Microservices register and deregister with a service registry.
- Clients resolve services by name, not IP — IPs come and go.
- Fan-out and loose coupling are key cloud-native design patterns.

Free tools used

- HashiCorp Consul — https://www.consul.io
- Docker — https://www.docker.com

Lab 6 — Containerization with Docker

Goal: Practise every core containerization sub-objective — build/run/expose, ephemeral vs persistent storage, public/private registries and a Compose preview.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

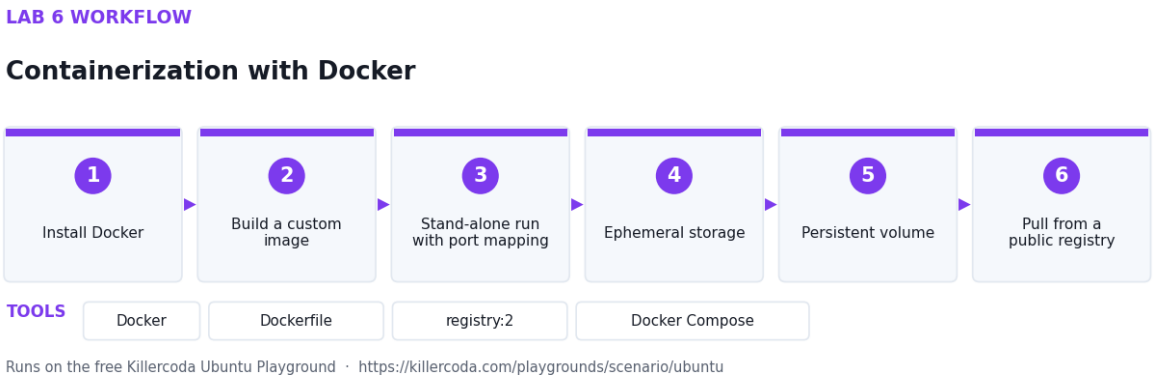

Figure 6 — the workflow you build in this lab.

Runs on: Docker Desktop — https://www.docker.com/products/docker-desktop/

Tools used: Docker, Dockerfile, registry:2, Docker Compose

In this lab you will build a custom image, run a stand-alone container, expose ports, mount persistent and ephemeral storage, and push to a local registry. By the end you will have practised every CV0-004 containerization sub-objective.

Lab platform

Run this lab either way:

- **Killercoda Ubuntu Playground** (nothing to install) — https://killercoda.com/playgrounds/scenario/ubuntu
- **Docker Desktop** on your own machine — https://www.docker.com/products/docker-desktop/

> *On Docker Desktop drop the `apt install docker.io` and `systemctl start docker` steps — the engine is already running. Everything else is identical.*

Step 1 — Install Docker

```
apt update && apt install -y docker.io
systemctl start docker
docker --version
```

Step 2 — Build a custom image

> *Ready-made files: `index.html` and `Dockerfile` — you can download them instead of typing this block.*

```
mkdir -p /tmp/app && cd /tmp/app
cat > index.html <<'EOF'
<h1>Cloud+ container</h1>
EOF
cat > Dockerfile <<'EOF'
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
EOF

docker build -t myapp:1.0 .
docker images | grep myapp
```

Step 3 — Stand-alone run with port mapping

```
docker run -d --name web -p 8080:80 myapp:1.0
curl -s http://localhost:8080
```

`-p 8080:80` maps host port 8080 → container port 80. This is **port mapping**, a CV0-004 sub-objective.

Step 4 — Ephemeral storage (default)

```
docker exec web sh -c 'echo lost > /tmp/scratch.txt'
docker exec web cat /tmp/scratch.txt
docker rm -f web
docker run -d --name web -p 8080:80 myapp:1.0
docker exec web ls /tmp/scratch.txt 2>&1 || echo "Gone — ephemeral"
```

The file disappeared with the container — that is **ephemeral storage**.

Step 5 — Persistent volume

```
docker volume create app-data
docker rm -f web
docker run -d --name web -p 8080:80 \
  -v app-data:/data myapp:1.0
docker exec web sh -c 'echo durable > /data/keep.txt'
docker rm -f web
docker run --rm -v app-data:/data alpine cat /data/keep.txt
```

The file survived a container delete — that is a **persistent volume**.

Step 6 — Pull from a public image registry

```
docker pull alpine:3.20
docker pull hello-world
docker images
```

`docker.io` is a **public** registry. You can also push to private registries like ECR, ACR, GAR.

Step 7 — Run a private registry

```
docker run -d --name registry -p 5000:5000 registry:2
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
docker pull localhost:5000/myapp:1.0
curl -s http://localhost:5000/v2/_catalog
```

Step 8 — Workload orchestration preview (Compose)

> *Ready-made file: `docker-compose.yml` — you can download it instead of typing this block.*

```
apt install -y docker-compose-v2 || apt install -y docker-compose
cat > /tmp/docker-compose.yml <<'EOF'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
services:
  web:
    image: myapp:1.0
    ports: ["8081:80"]
  cache:
    image: redis:7-alpine
EOF
cd /tmp && docker compose up -d
docker compose ps
```

This previews **workload orchestration** — covered fully in Lab 32 (Kubernetes).

Step 9 — Cleanup

```
cd / && docker compose -f /tmp/docker-compose.yml down
docker rm -f web registry
docker volume rm app-data
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker images | grep -E 'myapp|alpine|hello-world'
curl -s http://localhost:8080
docker run --rm -v app-data:/data alpine cat /data/keep.txt
curl -s http://localhost:5000/v2/_catalog
docker compose -f /tmp/docker-compose.yml ps
```

Expected: Run this before Step 9. `myapp:1.0` (plus `alpine:3.20` and `hello-world`) appear in the image list; `curl` on port 8080 returns `<h1>Cloud+ container</h1>`; the volume read prints `durable`, proving the named volume outlived the container; the local registry catalog returns `{"repositories":["myapp"]}`; and `docker compose ps` shows the `web` and `cache` services running.

What you learned

- Build, tag, run, and push images.
- Port mapping vs persistent volumes vs ephemeral storage.
- Public vs private image registries.

Free tools used

- Docker Engine — https://www.docker.com
- Docker Hub — https://hub.docker.com
- Distribution registry — https://github.com/distribution/distribution

Files in this lab

File	Purpose
`index.html`	Step 2 page baked into the custom `myapp:1.0` image.
`Dockerfile`	Step 2 image definition — nginx:alpine plus the custom index page.
`docker-compose.yml`	Step 8 Compose stack — web + redis cache orchestration preview.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 7 — Virtualization with QEMU/KVM

Goal: Create and clone a QEMU VM to understand core virtualization concepts and contrast VMs with containers.

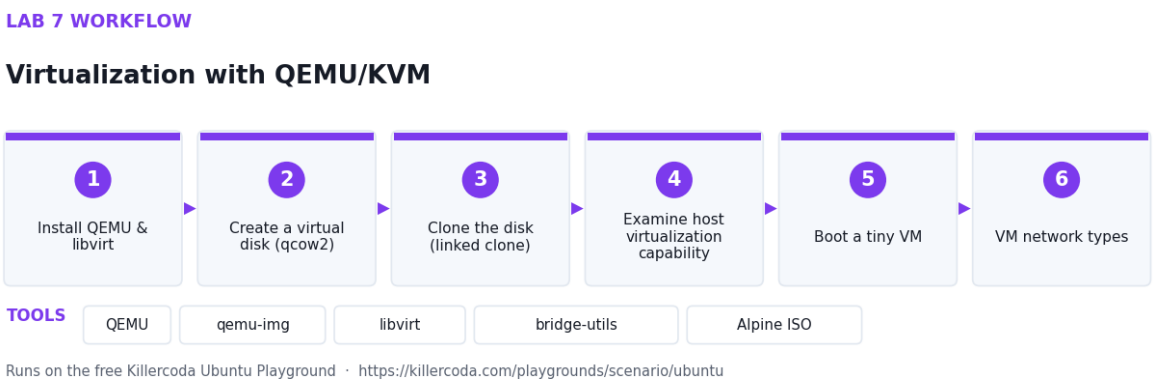

Figure 7 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: QEMU, qemu-img, libvirt, bridge-utils, Alpine ISO

In this lab you will create a virtual machine with QEMU, examine virtualization concepts (clone, host affinity, hardware pass-through, VM networks), and compare it to the container model from Lab 6.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

> *Note: Killercoda runs nested under KVM, so we use **QEMU TCG** (software emulation) instead of full KVM acceleration. The concepts and commands are identical to a real hypervisor.*

Step 1 — Install QEMU and libvirt tools

```
apt update && apt install -y qemu-system-x86 qemu-utils libvirt-clients
libvirt-daemon-system bridge-utils virtinst cloud-image-utils
```

Step 2 — Create a virtual disk (VM storage, local)

```
mkdir -p /var/vm && cd /var/vm
qemu-img create -f qcow2 vm1.qcow2 2G
qemu-img info vm1.qcow2
```

`qcow2` is a copy-on-write disk format — the basis of fast **cloning**.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 3 — Clone the disk (linked clone)

```
qemu-img create -f qcow2 -F qcow2 -b vm1.qcow2 vm2.qcow2
ls -lh vm*.qcow2
qemu-img info vm2.qcow2
```

`vm2.qcow2` is a thin clone — only stores deltas. This is how cloud snapshots and templates work.

Step 4 — Examine the host's virtualization capability

```
egrep -c '(vmx|svm)' /proc/cpuinfo
lscpu | grep -i virtual
```

If `vmx` (Intel) or `svm` (AMD) is non-zero, the host supports hardware-assisted virtualization (KVM). That is **hardware pass-through** at the CPU level.

Step 5 — Boot a tiny VM

```
cd /var/vm
wget -q https://dl-cdn.alpinelinux.org/alpine/v3.19/releases/x86_64/alpine-
virt-3.19.1-x86_64.iso

timeout 30 qemu-system-x86_64 \
  -m 256 \
  -nographic \
  -hda vm1.qcow2 \
  -cdrom alpine-virt-3.19.1-x86_64.iso \
  -boot d \
  -net nic -net user 2>&1 | head -30 || true
```

The VM boots its own kernel — unlike a container, which shares the host kernel.

Step 6 — VM network types

QEMU shows the two main virtual network types from the exam:

```
echo "--- User-mode (NAT, default) ---"
echo '-net user creates an isolated NAT — like an overlay network'

echo "--- Bridged (VM joins host LAN) ---"
brctl addbr br0 2>/dev/null || ip link add br0 type bridge
ip link show br0
ip link del br0
```

- **Overlay** = `-net user` — VM traffic encapsulated, NATed.
- **VM network (bridged)** = bridge — VMs appear on the same L2 segment as the host.

Step 7 — Host affinity

In libvirt you pin a VM to a specific host CPU set:

```
echo '<vcpu placement="static" cpuset="0-1">2</vcpu>'
```

`cpuset` = host affinity. The same concept on AWS is **Dedicated Host** + placement groups.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 8 — VM vs Container summary

Aspect	VM (this lab)	Container (Lab 6)
Kernel	Own kernel	Shares host kernel
Boot time	seconds–minutes	milliseconds
Disk	qcow2 / raw	layered images
Isolation	Hardware-level	Namespace + cgroup
Density per host	Tens	Thousands

Step 9 — Cleanup

```
rm -f /var/vm/vm*.qcow2 /var/vm/alpine-virt-3.19.1-x86_64.iso
```

Test it

Run these checks to prove the lab worked before you move on:

```
qemu-img info /var/vm/vm1.qcow2
qemu-img info /var/vm/vm2.qcow2 | grep -i backing
ls -lh /var/vm/vm*.qcow2
egrep -c '(vmx|svm)' /proc/cpuinfo
```

Expected: Run this before Step 9. `vm1.qcow2` reports `file format: qcow2` with a 2.0 GiB virtual size; `vm2.qcow2` names `vm1.qcow2` as its **backing file** and its on-disk size is only a few hundred KB versus vm1's — proof the linked clone stores deltas only; the CPU flag count is a number (non-zero means the host supports hardware-assisted virtualization).

What you learned

- Create and clone qcow2 disks.
- VM networks: user-mode, bridged, overlay.
- VMs vs containers — when to choose which.

Free tools used

- QEMU — https://www.qemu.org
- libvirt / virt-install — https://libvirt.org
- Alpine Linux ISO — https://alpinelinux.org

Lab 8 — Relational vs Non-Relational Databases

Goal: Run and compare a self-managed relational DB against a document DB to learn when to choose each and how managed services shift responsibility.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

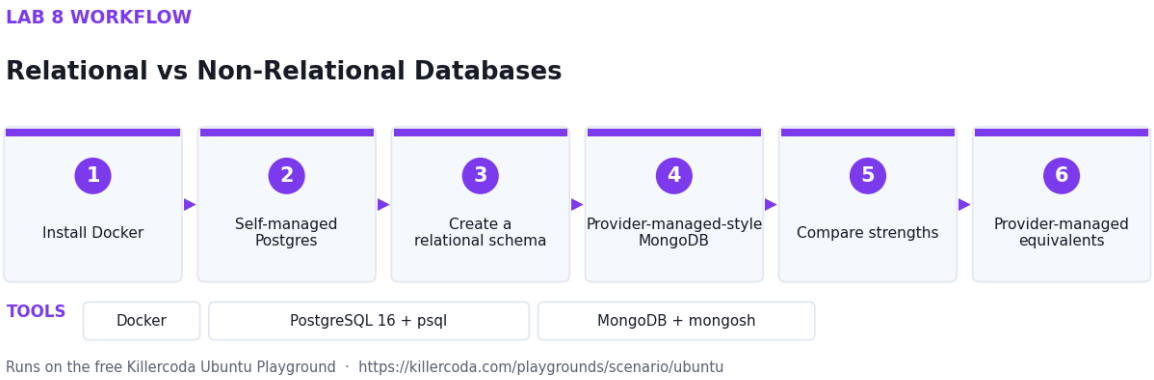

Figure 8 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Docker, PostgreSQL 16 + psql, MongoDB + mongosh

In this lab you will run a **self-managed** Postgres (relational) and a **provider-managed-style** MongoDB (non-relational), compare schemas and queries, and discuss when to choose each.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Docker

```
apt update && apt install -y docker.io postgresql-client
systemctl start docker
```

Step 2 — Self-managed Postgres (you handle everything)

```
docker run -d --name pg \
  -e POSTGRES_PASSWORD=cloud \
  -p 5432:5432 \
  postgres:16

sleep 5
PGPASSWORD=cloud psql -h 127.0.0.1 -U postgres -c "SELECT version();"
```

You picked the version, set the password, manage backups. **Self-managed** = customer-owned lifecycle.

Step 3 — Create a relational schema

```
PGPASSWORD=cloud psql -h 127.0.0.1 -U postgres <<'SQL'
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  name TEXT
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
);
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  customer_id INT REFERENCES customers(id),
  amount NUMERIC(10,2)
);
INSERT INTO customers(email, name) VALUES ('a@x.com','Alice'),
('b@x.com','Bob');
INSERT INTO orders(customer_id, amount) VALUES (1, 12.50), (1, 99.00), (2,
5.00);

SELECT c.name, SUM(o.amount) AS total
FROM customers c JOIN orders o ON o.customer_id = c.id
GROUP BY c.name;
SQL
```

Strict schema, foreign keys, JOIN — relational strengths.

Step 4 — Provider-managed-style MongoDB

```
docker run -d --name mongo -p 27017:27017 mongo:7
sleep 5
docker exec -i mongo mongosh <<'JS'
use shop
db.customers.insertOne({email:"a@x.com", name:"Alice", orders:[
  {amount: 12.50, item:"pen"},
  {amount: 99.00, item:"chair"}
]})
db.customers.insertOne({email:"b@x.com", name:"Bob", orders:[
  {amount: 5.00, item:"pad"}
]})

db.customers.aggregate([
  {$unwind:"$orders"},
  {$group:{_id:"$name", total:{$sum:"$orders.amount"}}}
])
JS
```

No schema enforced, nested documents, no JOIN. Suitable for variable-shape data.

Step 5 — Compare strengths

Need	Relational (Postgres)	Non-relational (Mongo)
Strict schema, ACID	✅	partial
Multi-table JOIN	✅	❌
Variable structure	❌	✅
Horizontal scale-out	harder	built-in sharding
Examples	Banking, ERP	IoT, catalog, sessions

Step 6 — Provider-managed equivalents

Engine	AWS	Azure	GCP
Postgres	RDS / Aurora	Azure Database for Postgres	Cloud SQL
MongoDB	DocumentDB	Cosmos DB (Mongo API)	Firestore

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Redis	ElastiCache	Azure Cache for Redis	Memorystore

In **provider-managed** the cloud handles patching, HA, backups; you only manage data and connections.

Step 7 — Cleanup

```
docker rm -f pg mongo
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker ps --filter name=pg --filter name=mongo --format '{{.Names}}
\t{{.Status}}'
PGPASSWORD=cloud psql -h 127.0.0.1 -U postgres -c "\dt"
PGPASSWORD=cloud psql -h 127.0.0.1 -U postgres -c "SELECT c.name,
SUM(o.amount) FROM customers c JOIN orders o ON o.customer_id=c.id GROUP BY
c.name;"
docker exec -i mongo mongosh --quiet --eval
'db.getSiblingDB("shop").customers.countDocuments({})'
```

Expected: Run this before Step 7. Both `pg` and `mongo` are **Up**; `\dt` lists the `customers` and `orders` tables; the JOIN returns `Alice | 111.50` and `Bob | 5.00`; and MongoDB reports `2` customer documents — the same business data expressed relationally and as nested documents.

What you learned

- Relational vs non-relational data models.
- Self-managed vs provider-managed responsibility split.
- When to pick each engine.

Free tools used

- PostgreSQL — https://www.postgresql.org
- MongoDB Community — https://www.mongodb.com/try/download/community
- DB Fiddle (web SQL sandbox) — https://www.db-fiddle.com

Domain 2 — Deployment (19%)

Provisioning and configuring cloud resources: deployment models, zero-downtime release strategies (blue-green, canary, rolling), Infrastructure as Code, Configuration as Code and scripting logic.

What this domain covers in the labs:

- **Lab 9 — Public, Private & Hybrid Cloud Models:** Simulate public (LocalStack), private (MinIO) and hybrid deployment models to compare their cost, control and elasticity trade-offs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Lab 10 — Blue-Green Deployment with Nginx:** Run blue (v1) and green (v2) side by side and switch live traffic instantly via an Nginx reload to achieve zero-downtime deployment.
- **Lab 11 — Canary Deployment with HAProxy:** Gradually shift live traffic from v1 to v2 using HAProxy server weights to safely validate a new release before full promotion.
- **Lab 12 — Rolling Deployment with Docker Compose:** Deploy three load-balanced replicas and upgrade them one at a time to perform a zero-downtime rolling update.
- **Lab 13 — Infrastructure as Code with Terraform:** Write a Terraform config that provisions an S3 bucket on LocalStack and experience plan/apply, state, drift detection and versioning.
- **Lab 14 — Configuration as Code with Ansible:** Write an Ansible playbook that installs and configures Nginx idempotently and prove drift correction.
- **Lab 15 — JSON & YAML Scripting Logic:** Parse, transform, validate and convert JSON ↔ YAML while exercising the exam's scripting-logic primitives.

Lab 9 — Public, Private & Hybrid Cloud Models

Goal: Simulate public (LocalStack), private (MinIO) and hybrid deployment models to compare their cost, control and elasticity trade-offs.

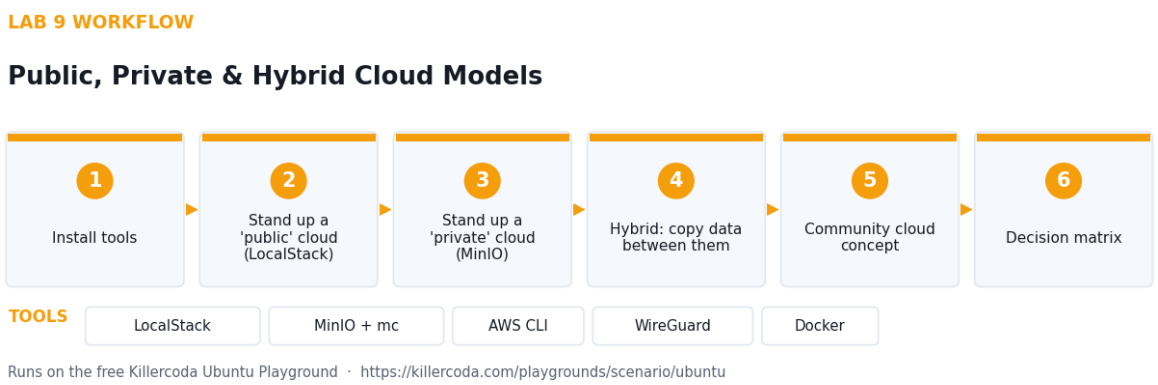

Figure 9 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: LocalStack, MinIO + mc, AWS CLI, WireGuard, Docker

In this lab you will simulate a **private** cloud (your Killercoda VM), a **public** cloud (LocalStack — a free local AWS), and connect them as a **hybrid** with a WireGuard tunnel. By the end you will see why each model trades cost, control, and elasticity differently.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install tools

```
apt update && apt install -y docker.io awscli wireguard wireguard-tools
systemctl start docker
```

Step 2 — Stand up a "public" cloud with LocalStack

LocalStack is a free, open-source AWS emulator — perfect for the lab without a real account.

```
docker run -d --name public-cloud -p 4566:4566 \
  -e SERVICES=s3,ec2,iam \
  localstack/localstack:latest
sleep 8
curl -s http://localhost:4566/_localstack/health | head -c 200
```

Configure the AWS CLI to talk to it:

```
aws configure set aws_access_key_id test
aws configure set aws_secret_access_key test
aws configure set default.region us-east-1

aws --endpoint-url=http://localhost:4566 s3 mb s3://public-bucket
aws --endpoint-url=http://localhost:4566 s3 ls
```

This is your **public cloud** — multi-tenant API, region-based, pay-per-use.

Step 3 — Stand up a "private" cloud (on-prem)

Use a local MinIO bucket — the same S3 API, but inside your data centre.

```
docker run -d --name private-cloud -p 9000:9000 \
  -e MINIO_ROOT_USER=admin -e MINIO_ROOT_PASSWORD=cloudplus \
  minio/minio server /data

sleep 4
docker exec private-cloud mc alias set local http://127.0.0.1:9000 admin
cloudplus
docker exec private-cloud mc mb local/private-bucket
```

This is **private** / on-prem — single tenant, full control.

Step 4 — Hybrid: copy data between them

```
echo "hybrid-payload" > /tmp/data.txt
aws --endpoint-url=http://localhost:4566 s3 cp /tmp/data.txt s3://public-
bucket/
docker cp /tmp/data.txt private-cloud:/data.txt
docker exec private-cloud mc cp /data.txt local/private-bucket/

aws --endpoint-url=http://localhost:4566 s3 ls s3://public-bucket/
docker exec private-cloud mc ls local/private-bucket/
```

The same object now lives in both clouds — the basis of a **hybrid** model.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 5 — Community cloud concept

A community cloud is shared by a group with common compliance needs (e.g. healthcare, government). You would model it as a private cloud with **multi-tenant access** restricted to the community. No infra to deploy — just policy.

Step 6 — Decision matrix

Factor	Public	Private	Hybrid	Community
CapEx	Low	High	Mixed	Shared
Elasticity	Best	Limited	Burst-able	Limited
Control	Lowest	Highest	Mixed	Shared
Compliance	varies	strongest	depends	strongest for sector

Step 7 — Cleanup

```
docker rm -f public-cloud private-cloud
```

Test it

Run these checks to prove the lab worked before you move on:

```
curl -s http://localhost:4566/_localstack/health | head -c 200
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 s3 ls s3://public-bucket/
docker exec private-cloud mc ls local/private-bucket/
```

Expected: Run this before Step 7. LocalStack's health endpoint reports `s3` as `"available"` or `"running"`; `s3 ls` lists `public-bucket`; and **the same object `data.txt` appears in both** the LocalStack `public-bucket` listing and the MinIO `private-bucket` listing — the hybrid copy succeeded across the two clouds.

What you learned

- Public, private, hybrid, and community models differ in tenancy and control.
- The **API** can stay the same (S3) across deployment models.
- Hybrid lets you burst from on-prem to public cloud.

Free tools used

- LocalStack Community — https://www.localstack.cloud
- MinIO — https://min.io
- AWS CLI — https://aws.amazon.com/cli

Lab 10 — Blue-Green Deployment with Nginx

Goal: Run blue (v1) and green (v2) side by side and switch live traffic instantly via an Nginx reload to achieve zero-downtime deployment.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Figure 10 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Nginx, Docker (nginx:alpine), curl

In this lab you will run two versions of an application — the live "blue" v1 and the staged "green" v2 — and switch traffic instantly by reloading Nginx. This is the canonical zero-downtime deployment pattern.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Nginx and Docker

```
apt update && apt install -y nginx docker.io curl
systemctl start docker
systemctl start nginx
```

Step 2 — Run Blue (v1) and Green (v2)

```
docker run -d --name blue  -p 8081:80 nginx:alpine
docker run -d --name green -p 8082:80 nginx:alpine

docker exec blue  sh -c 'echo "v1 BLUE"  > /usr/share/nginx/html/index.html'
docker exec green sh -c 'echo "v2 GREEN" > /usr/share/nginx/html/index.html'
```

Step 3 — Point Nginx at Blue

> *Ready-made file: `nginx-blue-green.conf` — you can download it instead of typing this block.*

```
cat > /etc/nginx/sites-available/default <<'EOF'
upstream live { server 127.0.0.1:8081; }   # BLUE
server {
    listen 80;
    location / { proxy_pass http://live; }
}
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
EOF
nginx -t && systemctl reload nginx
curl -s http://localhost/
```

Step 4 — Smoke-test Green out of band

```
curl -s http://localhost:8082/
```

Green is fully running but receives **zero customer traffic**. Run integration tests here.

Step 5 — Switch traffic to Green (cutover)

```
sed -i 's/8081/8082/' /etc/nginx/sites-available/default
nginx -t && systemctl reload nginx
for i in 1 2 3; do curl -s http://localhost/; done
```

`nginx -s reload` swaps without dropping a connection. Cutover is **atomic**.

Step 6 — Instant rollback

If v2 is broken, switch back in one command:

```
sed -i 's/8082/8081/' /etc/nginx/sites-available/default
nginx -t && systemctl reload nginx
curl -s http://localhost/
```

This is why blue-green is preferred for high-risk releases.

Step 7 — Cleanup

```
docker rm -f blue green
systemctl stop nginx
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker ps --filter name=blue --filter name=green --format '{{.Names}}
\t{{.Status}}'
curl -s http://localhost:8081/
curl -s http://localhost:8082/
nginx -t
curl -s http://localhost/
```

Expected: Run this before Step 7. Both `blue` and `green` are **Up**; port 8081 returns `v1 BLUE` and port 8082 returns `v2 GREEN` (both environments live); `nginx -t` prints `syntax is ok` / `test is successful`; and port 80 returns whichever colour the upstream currently points at — `v1 BLUE` after the Step 6 rollback.

What you learned

- Blue-green keeps two parallel environments; cutover is a config change.
- Rollback is symmetrical and fast.
- Smoke-test the new version on its own port before flipping traffic.

Free tools used

- Nginx — https://nginx.org
- Docker — https://www.docker.com

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Files in this lab

File	Purpose
`nginx-blue-green.conf`	Step 3 Nginx site config — the `live` upstream that is flipped between blue (8081) and green (8082).

Lab 11 — Canary Deployment with HAProxy

Goal: Gradually shift live traffic from v1 to v2 using HAProxy server weights to safely validate a new release before full promotion.

Figure 11 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: HAProxy, Docker (nginx:alpine), curl

In this lab you will gradually shift traffic from v1 to v2 using HAProxy server weights — 90/10, then 50/50, then 100% — exactly the strategy used by AWS ALB weighted target groups, Istio canaries, and Argo Rollouts.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install HAProxy and Docker

```
apt update && apt install -y haproxy docker.io curl
systemctl start docker
```

Step 2 — Run v1 (stable) and v2 (canary)

```
docker run -d --name v1 -p 8091:80 nginx:alpine
docker run -d --name v2 -p 8092:80 nginx:alpine
docker exec v1 sh -c 'echo v1 > /usr/share/nginx/html/index.html'
docker exec v2 sh -c 'echo v2 > /usr/share/nginx/html/index.html'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 3 — HAProxy at 90/10 (canary 10%)

Ready-made file: `haproxy-canary-90-10.cfg` — you can download it instead of typing this block.

```
cat > /etc/haproxy/haproxy.cfg <<'EOF'
global
    daemon
defaults
    mode http
    timeout connect 5s
    timeout client 30s
    timeout server 30s
frontend fe
    bind *:80
    default_backend app
backend app
    balance roundrobin
    server v1 127.0.0.1:8091 weight 90 check
    server v2 127.0.0.1:8092 weight 10 check
EOF
systemctl restart haproxy
```

Verify the split:

```
for i in $(seq 1 50); do curl -s http://localhost/; done | sort | uniq -c
```

You should see ~45 v1 / ~5 v2.

Step 4 — Increase to 50/50

```
sed -i 's/weight 90/weight 50/; s/weight 10/weight 50/'
/etc/haproxy/haproxy.cfg
systemctl reload haproxy

for i in $(seq 1 50); do curl -s http://localhost/; done | sort | uniq -c
```

Step 5 — Promote v2 to 100%

Ready-made file: `haproxy-promote-v2.cfg` — you can download it instead of typing this block.

```
sed -i 's/weight 50/weight 0/; 0,/weight 0/!s/weight 0/weight 100/'
/etc/haproxy/haproxy.cfg
# Simpler: rewrite cleanly
cat > /etc/haproxy/haproxy.cfg <<'EOF'
global
    daemon
defaults
    mode http
    timeout connect 5s
    timeout client 30s
    timeout server 30s
frontend fe
    bind *:80
    default_backend app
backend app
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    server v1 127.0.0.1:8091 weight 0  check backup
    server v2 127.0.0.1:8092 weight 100 check
EOF
systemctl reload haproxy

for i in $(seq 1 20); do curl -s http://localhost/; done | sort | uniq -c
```

100% v2. Roll-out complete.

Step 6 — Rollback (kill the canary)

If your monitoring (Lab 16) shows v2 errors rising, flip the weights back:

```
sed -i 's/weight 0  check backup/weight 100 check/; s/weight 100 check$/weight
0  check backup/' /etc/haproxy/haproxy.cfg
systemctl reload haproxy
curl -s http://localhost/
```

Step 7 — Cleanup

```
docker rm -f v1 v2
systemctl stop haproxy
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker ps --filter name=v1 --filter name=v2 --format '{{.Names}}\t{{.Status}}'
systemctl is-active haproxy
grep -E 'server v[12]' /etc/haproxy/haproxy.cfg
for i in $(seq 1 20); do curl -s http://localhost/; done | sort | uniq -c
```

Expected: Run this before Step 7. Both `v1` and `v2` containers are **Up** and `haproxy` is `active`; the config shows the current weights for each server; and the 20-request tally matches those weights — roughly 18 `v1` / 2 `v2` at 90/10, about 10/10 at 50/50, and `20 v2` once v2 is promoted to 100%.

What you learned

- Canary = small percentage of real traffic on a new version.
- Weight-based routing lets you ramp gradually and rollback instantly.
- Always pair canary with metrics/alerts.

Free tools used

- HAProxy — https://www.haproxy.org
- Docker — https://www.docker.com

Files in this lab

File	Purpose
`haproxy-canary-90-10.cfg`	Step 3 HAProxy config — 90/10 weighted split between v1 and the v2 canary.
`haproxy-promote-v2.cfg`	Step 5 HAProxy config — v2 promoted to 100%, v1 kept as backup.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 12 — Rolling Deployment with Docker Compose

Goal: Deploy three load-balanced replicas and upgrade them one at a time to perform a zero-downtime rolling update.

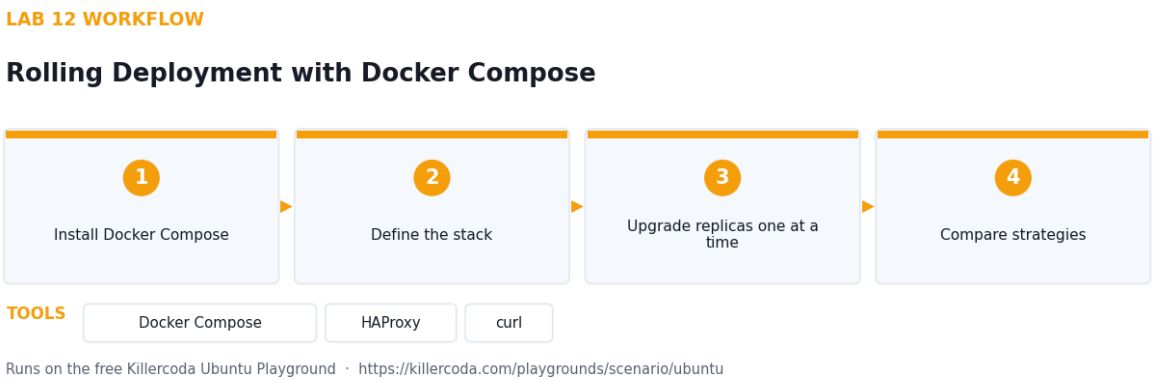

Figure 12 — the workflow you build in this lab.

Runs on: Docker Desktop — https://www.docker.com/products/docker-desktop/

Tools used: Docker Compose, HAProxy, curl

In this lab you will deploy three replicas of an app behind a load balancer, then upgrade them **one at a time** while traffic continues — the rolling deployment strategy used by Kubernetes Deployments, ECS services, and ASGs with rolling updates.

Lab platform

Run this lab either way:

- **Killercoda Ubuntu Playground** (nothing to install) — https://killercoda.com/playgrounds/scenario/ubuntu
- **Docker Desktop** on your own machine — https://www.docker.com/products/docker-desktop/

> *On Docker Desktop drop the `apt install docker.io` and `systemctl start docker` steps — the engine is already running. Everything else is identical.*

Step 1 — Install Docker Compose

```
apt update && apt install -y docker.io docker-compose-v2 curl
systemctl start docker
```

Step 2 — Define the stack

> *Ready-made files: `docker-compose.yml` and `haproxy.cfg` — you can download them instead of typing this block.*

```
mkdir -p /tmp/rolling && cd /tmp/rolling
cat > docker-compose.yml <<'EOF'
services:
  app:
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    image: nginx:alpine
    deploy:
      replicas: 3
    networks: [back]
    command: >
      sh -c 'echo "v1 from $$HOSTNAME" > /usr/share/nginx/html/index.html &&
nginx -g "daemon off;"'
  lb:
    image: haproxy:alpine
    ports: ["80:80"]
    volumes: ["./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro"]
    networks: [back]
networks:
  back:
EOF

cat > haproxy.cfg <<'EOF'
defaults
  mode http
  timeout connect 3s
  timeout client 30s
  timeout server 30s
frontend fe
  bind *:80
  default_backend app
backend app
  balance roundrobin
  server-template app- 3 app:80 check resolvers docker init-addr libc,none
resolvers docker
  nameserver dns 127.0.0.11:53
EOF

docker compose up -d --scale app=3
sleep 4
for i in 1 2 3 4 5 6; do curl -s http://localhost/; done
```

Step 3 — Upgrade replicas one at a time

```
# Replica 1
CID=$(docker compose ps -q app | head -n1)
docker stop $CID && docker rm $CID
docker compose up -d --scale app=3 --no-recreate
sleep 2

# Replica 2
CID=$(docker compose ps -q app | sed -n 2p)
docker stop $CID && docker rm $CID
docker compose up -d --scale app=3 --no-recreate
sleep 2

# Replica 3
CID=$(docker compose ps -q app | tail -n1)
docker stop $CID && docker rm $CID
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker compose up -d --scale app=3 --no-recreate
sleep 2
```

While each is replaced, the other two keep serving — no downtime.

```
for i in $(seq 1 9); do curl -s http://localhost/; done
```

Step 4 — Compare strategies

Strategy	Resource use	Risk window	Rollback
In-place	1×	Total downtime during swap	Restore + redeploy
Rolling	1×–1.x×	Mixed versions during roll	Roll backwards
Blue-Green	2×	Zero (atomic flip)	Instant
Canary	1×–1.x×	Small % users	Reset weights

Step 5 — Cleanup

```
cd /tmp/rolling && docker compose down
```

Test it

Run these checks to prove the lab worked before you move on:

```
cd /tmp/rolling && docker compose ps
docker compose ps -q app | wc -l
for i in $(seq 1 9); do curl -s http://localhost/; done
docker compose logs lb --tail 5
```

Expected: Run this before Step 5. `docker compose ps` shows the `lb` service plus the app replicas all in state **running**, the replica count is exactly `3`, and the nine `curl` calls return `v1 from <hostname>` cycling through three different container hostnames — proving HAProxy still balanced across surviving replicas while each one was replaced.

What you learned

- Rolling = replace replicas one (or batch) at a time.
- Cheaper than blue-green but produces a brief mixed-version window.
- Always pair with a readiness check before progressing.

Free tools used

- Docker Compose — https://docs.docker.com/compose
- HAProxy — https://www.haproxy.org

Files in this lab

File	Purpose
`docker-compose.yml`	Step 2 Compose stack — 3 app replicas behind an HAProxy load balancer.
`haproxy.cfg`	Step 2 HAProxy config — server-template with Docker DNS resolvers for the replicas.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 13 — Infrastructure as Code with Terraform

Goal: Write a Terraform config that provisions an S3 bucket on LocalStack and experience plan/apply, state, drift detection and versioning.

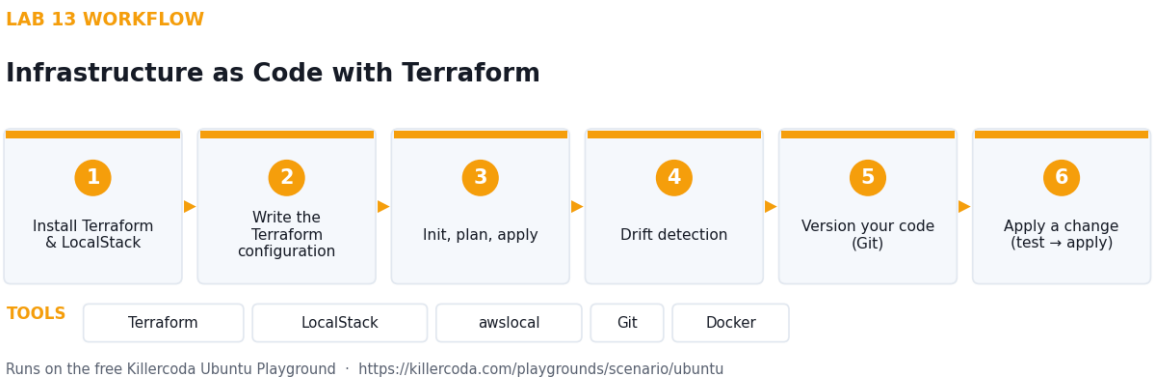

Figure 13 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Terraform, LocalStack, awslocal, Git, Docker

In this lab you will write a Terraform configuration that provisions an S3 bucket and a security policy on **LocalStack** (free local AWS). You will see plan/apply, state, drift detection, and versioning.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Terraform and LocalStack

```
apt update && apt install -y wget unzip docker.io
systemctl start docker

wget -q
https://releases.hashicorp.com/terraform/1.9.5/terraform_1.9.5_linux_amd64.zip
unzip -o terraform_1.9.5_linux_amd64.zip -d /usr/local/bin
terraform version

docker run -d --name localstack -p 4566:4566 -e SERVICES=s3,iam
localstack/localstack:latest
sleep 8
```

Step 2 — Write the Terraform configuration

Ready-made file: `main.tf` — you can download it instead of typing this block.

```
mkdir -p /tmp/tf && cd /tmp/tf
cat > main.tf <<'EOF'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
terraform {
  required_providers { aws = { source = "hashicorp/aws" version = "~> 5.0" } }
}

provider "aws" {
  region                      = "us-east-1"
  access_key                  = "test"
  secret_key                  = "test"
  skip_credentials_validation = true
  skip_metadata_api_check     = true
  skip_requesting_account_id  = true
  s3_use_path_style           = true
  endpoints { s3 = "http://localhost:4566" iam = "http://localhost:4566" }
}

variable "env" { default = "dev" }

resource "aws_s3_bucket" "data" {
  bucket = "cloudplus-${var.env}-data"
}
EOF
```

Step 3 — Init, plan, apply

```
terraform init
terraform plan
terraform apply -auto-approve
```

Inspect state:

```
terraform state list
terraform state show aws_s3_bucket.data
ls -lh terraform.tfstate
```

The state file is the source of truth — back it up to a remote backend in production.

Step 4 — Drift detection

Manually mutate the bucket outside Terraform:

```
docker exec localstack awslocal s3api put-bucket-tagging \
  --bucket cloudplus-dev-data \
  --tagging 'TagSet=[{Key=DriftedBy,Value=human}]'

terraform plan
```

`terraform plan` reports the **drift** — Terraform wants to remove the manual tag.

Step 5 — Version your code (Git)

```
apt install -y git
cd /tmp/tf
git init -q
echo 'terraform.tfstate*' > .gitignore
echo '.terraform/' >> .gitignore
git add . && git -c user.email=lab@x -c user.name=lab commit -q -m "initial
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
infra"
git log --oneline
```

Code reviewable, diff-able, rollback-able.

Step 6 — Apply a change (test → apply pattern)

```
sed -i 's/env" { default = "dev"/env" { default = "prod"/' main.tf
terraform plan
terraform apply -auto-approve
terraform state list
```

Terraform creates `cloudplus-prod-data`. The dev one is destroyed because the resource name is parameterised — typical IaC behaviour.

Step 7 — Cleanup

```
terraform destroy -auto-approve
docker rm -f localstack
```

Test it

Run these checks to prove the lab worked before you move on:

```
cd /tmp/tf && terraform state list
terraform state show aws_s3_bucket.data | head -10
terraform plan -detailed-exitcode
docker exec localstack awslocal s3 ls
git -C /tmp/tf log --oneline
```

Expected: Run this before Step 7. `terraform state list` prints `aws_s3_bucket.data`; `state show` reports the bucket name (`cloudplus-prod-data` after Step 6); `terraform plan` reports **No changes. Your infrastructure matches the configuration** (exit code 0) once applied, or lists the drifted tag if you run it before re-applying; `awslocal s3 ls` shows the bucket really exists in LocalStack; and `git log` shows the `initial infra` commit.

What you learned

- IaC: declarative config → cloud resources.
- Plan before apply.
- Drift detection catches manual changes.
- State must be stored safely.

Free tools used

- Terraform — https://developer.hashicorp.com/terraform
- LocalStack Community — https://www.localstack.cloud
- OpenTofu (Terraform fork) — https://opentofu.org

Files in this lab

File	Purpose
`main.tf`	Step 2 Terraform config — LocalStack AWS provider plus the parameterised S3 bucket.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 14 — Configuration as Code with Ansible

Goal: Write an Ansible playbook that installs and configures Nginx idempotently and prove drift correction.

Figure 14 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Ansible, ansible-galaxy, YAML, curl

In this lab you will write a small Ansible playbook that installs and configures Nginx idempotently. Configuration as Code (CaC) is one of the CV0-004 deployment sub-objectives.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Ansible

```
apt update && apt install -y ansible curl
ansible --version
```

Step 2 — Inventory (just localhost)

Ready-made file: `inventory.ini` — you can download it instead of typing this block.

```
mkdir -p /tmp/ans && cd /tmp/ans
cat > inventory.ini <<'EOF'
[web]
localhost ansible_connection=local
EOF
```

Step 3 — Write a playbook (YAML)

Ready-made file: `site.yml` — you can download it instead of typing this block.

```
cat > site.yml <<'EOF'
---
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
- name: Configure web server
  hosts: web
  become: true
  vars:
    site_msg: "Configured by Ansible"
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
        update_cache: true

    - name: Deploy index page
      copy:
        dest: /var/www/html/index.nginx-debian.html
        content: "<h1>{{ site_msg }}</h1>"

    - name: Ensure nginx is running
      service:
        name: nginx
        state: started
        enabled: true
EOF
```

Step 4 — First run (changes everything)

```
ansible-playbook -i inventory.ini site.yml
curl -s http://localhost
```

Look at the output: every task is `changed`.

Step 5 — Idempotency test (run again)

```
ansible-playbook -i inventory.ini site.yml
```

This time every task is `ok` — no work done. **Idempotency** is the core CaC property.

Step 6 — Detect drift / repeatability

Manually corrupt the page:

```
echo "drifted" > /var/www/html/index.nginx-debian.html
ansible-playbook -i inventory.ini site.yml
curl -s http://localhost
```

Ansible re-applies the desired state.

Step 7 — Variables and conditionals

Ready-made file: `extra.yml` — you can download it instead of typing this block.

```
cat > extra.yml <<'EOF'
---
- hosts: web
  become: true
  vars:
    enable_https: false
  tasks:
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    - name: Install certbot only if HTTPS requested
      apt:
        name: certbot
        state: present
      when: enable_https
EOF
ansible-playbook -i inventory.ini extra.yml --check
```

`when:` is a conditional. Re-run with `-e enable_https=true` to flip the path.

Step 8 — Roles preview

```
ansible-galaxy init /tmp/ans/roles/myweb
ls /tmp/ans/roles/myweb
```

Roles are reusable bundles — the equivalent of Terraform modules.

Step 9 — Cleanup

```
apt remove -y nginx
```

Test it

Run these checks to prove the lab worked before you move on:

```
cd /tmp/ans && ansible-playbook -i inventory.ini site.yml --check
systemctl is-active nginx
curl -s http://localhost
ls /tmp/ans/roles/myweb
```

Expected: Run this before Step 9. The `--check` dry run finishes with `changed=0` and `failed=0` in the PLAY RECAP — the host already matches the desired state, which is the proof of **idempotency**; `nginx` reports `active`; `curl` returns `<h1>Configured by Ansible</h1>` (Ansible re-applied the page after you drifted it); and the role skeleton lists `tasks`, `handlers`, `defaults`, `vars` and friends.

What you learned

- Playbooks are YAML.
- Idempotent tasks: only act if state differs.
- Variables, conditionals, and roles compose larger configurations.

Free tools used

- Ansible — https://www.ansible.com
- Ansible Galaxy — https://galaxy.ansible.com
- YAML Lint (web) — https://www.yamllint.com

Files in this lab

File	Purpose
`inventory.ini`	Step 2 Ansible inventory — a single local `web` host.
`site.yml`	Step 3 playbook — installs nginx, deploys the index page, ensures the service is running.
`extra.yml`	Step 7 playbook — variables and a `when:` conditional for optional certbot.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 15 — JSON & YAML Scripting Logic

Goal: Parse, transform, validate and convert JSON ↔ YAML while exercising the exam's scripting-logic primitives.

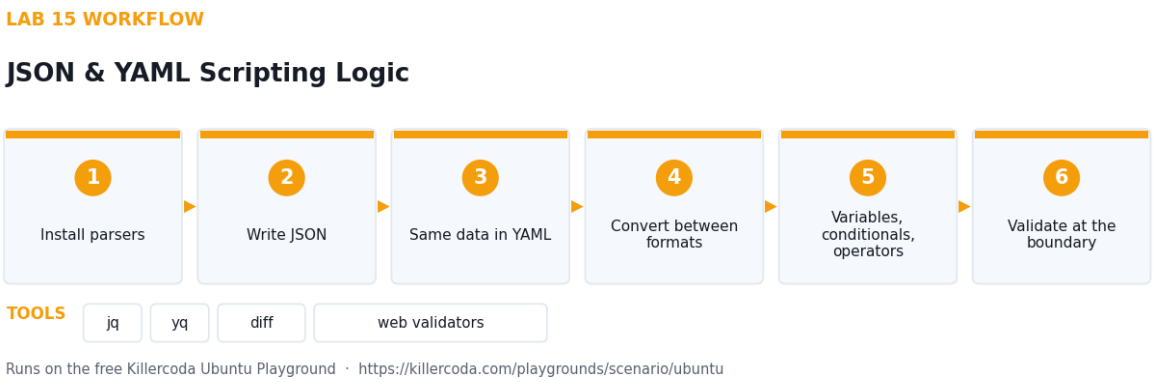

Figure 15 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: jq, yq, diff, web validators

In this lab you will read and write JSON and YAML, parse with `jq` and `yq`, and exercise the scripting-logic primitives the exam lists: variables, conditionals, operators, data types, functions.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install parsers

```
apt update && apt install -y jq python3-pip
pip3 install yq
```

Step 2 — Write JSON

Ready-made file: `server.json` — you can download it instead of typing this block.

```
mkdir -p /tmp/data && cd /tmp/data
cat > server.json <<'EOF'
{
  "name": "web-1",
  "cpu": 2,
  "ram_gb": 4,
  "tags": ["web", "prod"],
  "active": true
}
EOF
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
jq . server.json
jq '.name, .cpu' server.json
jq '.tags | length' server.json
```

Data types in play: string, number, boolean, array, object.

Step 3 — Same data in YAML

Ready-made file: `server.yaml` — you can download it instead of typing this block.

```
cat > server.yaml <<'EOF'
name: web-1
cpu: 2
ram_gb: 4
tags:
  - web
  - prod
active: true
EOF

yq . server.yaml
yq -y '.tags' server.yaml
```

Step 4 — Convert between formats

```
yq . server.yaml > /tmp/data/server-from-yaml.json
diff <(jq -S . server.json) <(jq -S . server-from-yaml.json) && echo "Same
data, two formats"
```

Step 5 — Variables, conditionals, operators

`jq` is itself a tiny scripting language — perfect to exercise the exam primitives.

```
# Variables
jq --arg env prod '. + {env:$env}' server.json

# Conditionals
jq 'if .ram_gb < 8 then "undersized" else "ok" end' server.json

# Operators
jq '.cpu * 2' server.json

# Functions
jq 'def double(x): x*2; double(.cpu)' server.json
```

Step 6 — Validate at the boundary

Try a broken file:

```
echo '{ "broken": ' > bad.json
jq . bad.json || echo "Schema violation caught"
```

In production, fail builds early on invalid JSON/YAML — same idea as Terraform `validate` or `kubectl --dry-run`.

Step 7 — Web validators (offline-friendly bookmarks)

- JSONLint — https://jsonlint.com

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- YAML Lint — https://www.yamllint.com
- JSON Schema validator — https://www.jsonschemavalidator.net

These are useful when the exam shows you a malformed snippet.

Step 8 — Cleanup

```
rm -rf /tmp/data
```

Test it

Run these checks to prove the lab worked before you move on:

```
cd /tmp/data && jq . server.json
jq '.tags | length' server.json
yq . server.yaml
diff <(jq -S . server.json) <(jq -S . server-from-yaml.json) && echo "Same
data, two formats"
jq 'if .ram_gb < 8 then "undersized" else "ok" end' server.json
```

Expected: Run this before Step 8. `jq` pretty-prints the JSON without a parse error, the `tags` array length is `2`, `yq` renders the YAML as the same JSON object, the `diff` produces **no output** and prints `Same data, two formats`, and the conditional evaluates to `"undersized"` because `ram_gb` is 4.

What you learned

- Read, write, validate, and convert JSON ↔ YAML.
- Scripting logic primitives at the data-format level.
- When to choose JSON (machine-friendly) vs YAML (human-friendly).

Free tools used

- jq — https://jqlang.github.io/jq
- yq — https://kislyuk.github.io/yq
- JSONLint — https://jsonlint.com
- YAML Lint — https://www.yamllint.com

Files in this lab

File	Purpose
`server.json`	Step 2 JSON sample exercising string, number, boolean, array and object types.
`server.yaml`	Step 3 YAML sample — the same data in the human-friendly format.

Domain 3 — Operations (17%)

Running cloud systems day-to-day: observability (metrics, logs, traces), alerting, auto-scaling, backup & disaster recovery, and patching / lifecycle management.

What this domain covers in the labs:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Lab 16 — Observability with Prometheus & Grafana:** Scrape a node's metrics with Prometheus, visualise them in Grafana and trigger a CPU alert.
- **Lab 17 — Centralized Logging with the ELK Stack:** Run ELK, ship logs into it, and query and retain them through the Elasticsearch API.
- **Lab 18 — Distributed Tracing with Jaeger:** Run Jaeger, instrument a Python 'checkout' path with OpenTelemetry and view a distributed trace of parent/child spans.
- **Lab 19 — Horizontal Auto-Scaling Simulation:** Simulate triggered, scheduled and manual horizontal scaling using a Bash controller that watches CPU and adds/removes Docker replicas.
- **Lab 20 — Backup & Recovery with restic:** Perform encrypted full/incremental/differential backups to off-site S3 storage, then test recoverability and verify integrity.
- **Lab 21 — Patching & Lifecycle Management:** Simulate a cloud resource lifecycle from provision through minor patch, major upgrade, testing and clean decommissioning.

Lab 16 — Observability with Prometheus & Grafana

Goal: Scrape a node's metrics with Prometheus, visualise them in Grafana and trigger a CPU alert.

Figure 16 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Prometheus, Grafana, node-exporter, stress-ng, Docker

In this lab you will scrape a node's metrics with **Prometheus**, visualise them in **Grafana**, and trigger an **alert** when CPU rises — covering the exam's logging/monitoring/alerting/triage sub-objectives.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Docker

```
apt update && apt install -y docker.io stress-ng curl
systemctl start docker
```

Step 2 — Run node-exporter (metrics source)

```
docker run -d --name node --net=host \
  prom/node-exporter:latest
sleep 2
curl -s http://localhost:9100/metrics | head -5
> Ready-made file: [`prom.yml`](prom.yml) — you can download it instead of
typing this block.
```

Step 3 — Configure and run Prometheus

Ready-made file: `alerts.yml` — you can download it instead of typing this block.

```
mkdir -p /tmp/prom && cat > /tmp/prom/prom.yml <<'EOF'
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: node
    static_configs:
      - targets: ['localhost:9100']
rule_files: ['alerts.yml']
EOF

cat > /tmp/prom/alerts.yml <<'EOF'
groups:
- name: cpu
  rules:
  - alert: HighCPU
    expr: 1 - avg(rate(node_cpu_seconds_total{mode="idle"}[1m])) > 0.5
    for: 30s
    labels: { severity: warning }
    annotations: { summary: "CPU > 50%" }
EOF

docker run -d --name prom --net=host \
  -v /tmp/prom:/etc/prometheus \
  prom/prometheus --config.file=/etc/prometheus/prom.yml
sleep 5
curl -s 'http://localhost:9090/api/v1/query?query=up' | head -c 200
```

Step 4 — Run Grafana

```
docker run -d --name grafana --net=host \
  -e GF_SECURITY_ADMIN_PASSWORD=cloud \
  grafana/grafana:latest
sleep 6
curl -s -u admin:cloud http://localhost:3000/api/health
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Add the Prometheus data source via API:

```
curl -s -u admin:cloud -H 'Content-Type: application/json' \
  -d
'{"name":"prom","type":"prometheus","url":"http://localhost:9090","access":"pr
oxy","isDefault":true}' \
  http://localhost:3000/api/datasources
```

Open `http://<killercoda-host>:3000` in the playground's "Traffic / Ports" tab and explore.

Step 5 — Trigger the alert

```
stress-ng --cpu 4 --timeout 60s &
sleep 45
curl -s 'http://localhost:9090/api/v1/alerts' | head -c 400
```

Within ~30s the `HighCPU` alert is **firing** — Prometheus does the triage; alertmanager (production) does the response (PagerDuty, Slack, Teams).

Step 6 — Three pillars summary

Pillar	Tool you used	Lab
Metrics	Prometheus + node-exporter	16
Logs	rsyslog / ELK	17
Traces	Jaeger	18

Together they make a system **observable** — you can ask new questions about it without re-deploying.

Step 7 — Cleanup

```
docker rm -f node prom grafana
```

Test it

Run these checks to prove the lab worked before you move on:

```
curl -s http://localhost:9100/metrics | head -5
curl -s 'http://localhost:9090/api/v1/targets' | jq
'.data.activeTargets[].health'
curl -s 'http://localhost:9090/api/v1/query?query=up' | jq
'.data.result[].value[1]'
curl -s -u admin:cloud http://localhost:3000/api/health
curl -s 'http://localhost:9090/api/v1/alerts' | jq '.data.alerts[].state'
```

Expected: Run this before Step 7. node-exporter returns Prometheus text metrics (`# HELP ...` lines); the Prometheus target health is `"up"` and the `up` query returns `"1"`; Grafana's health endpoint returns `"database": "ok"`; and while `stress-ng` is loading the CPU the alerts endpoint shows the `HighCPU` alert in state `"pending"` and then `"firing"` after ~30 seconds.

What you learned

- Scrape metrics, visualise, and alert on thresholds.
- Aggregation and retention are configured in Prometheus.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Alerting → triage → response is the operational loop.

Free tools used

- Prometheus — https://prometheus.io
- Grafana OSS — https://grafana.com/oss/grafana
- node-exporter — https://github.com/prometheus/node_exporter
- stress-ng — https://github.com/ColinIanKing/stress-ng

Files in this lab

File	Purpose
`prom.yml`	Step 3 Prometheus config — scrapes node-exporter every 5s and loads the alert rules.
`alerts.yml`	Step 3 alert rules — the `HighCPU` rule that fires above 50% CPU.

Lab 17 — Centralized Logging with the ELK Stack

Goal: Run ELK, ship logs into it, and query and retain them through the Elasticsearch API.

Figure 17 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Elasticsearch, Logstash, Kibana, Docker, curl, jq

In this lab you will run **Elasticsearch**, **Logstash**, and **Kibana** (ELK) on the Killercoda VM, ship logs from `rsyslog`, and query them through Kibana's API.

> *ELK images are heavy (~1.5 GB). On Killercoda, expect a slower pull. If memory runs tight, swap to the lighter **OpenSearch** alternative shown at the end.*

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

> ***Web tool:*** *Build and test the grok/regex patterns for Logstash with the free browser* ***Regex Generator*** *— https://alfredang.github.io/regexgenerator/ — before you put them in the pipeline config.*

Step 1 — Install Docker

```
apt update && apt install -y docker.io curl jq
systemctl start docker
sysctl -w vm.max_map_count=262144
```

Step 2 — Run Elasticsearch

```
docker run -d --name es --net=host \
  -e discovery.type=single-node \
  -e xpack.security.enabled=false \
  -e ES_JAVA_OPTS='-Xms512m -Xmx512m' \
  docker.elastic.co/elasticsearch/elasticsearch:8.13.0
sleep 30
curl -s http://localhost:9200/_cluster/health | jq
```

Step 3 — Run Kibana

```
docker run -d --name kibana --net=host \
  -e ELASTICSEARCH_HOSTS=http://localhost:9200 \
  docker.elastic.co/kibana/kibana:8.13.0
sleep 30
curl -s http://localhost:5601/api/status | head -c 200
```

Open `http://<killercoda-host>:5601` from the playground's Traffic tab.

Step 4 — Run Logstash with a syslog input

Ready-made file: `pipeline.conf` — you can download it instead of typing this block.

```
mkdir -p /tmp/ls && cat > /tmp/ls/pipeline.conf <<'EOF'
input { tcp { port => 5044 codec => line } }
filter { mutate { add_field => { "source" => "syslog" } } }
output { elasticsearch { hosts => ["http://localhost:9200"] index => "logs-%
{+YYYY.MM.dd}" } }
EOF

docker run -d --name logstash --net=host \
  -v /tmp/ls/pipeline.conf:/usr/share/logstash/pipeline/logstash.conf \
  docker.elastic.co/logstash/logstash:8.13.0
sleep 25
```

Step 5 — Ship test logs

```
for i in $(seq 1 20); do
  echo "$(date) cloudplus event $i severity=info" | nc -q 1 127.0.0.1 5044
done
sleep 3

curl -s 'http://localhost:9200/_cat/indices?v'
curl -s 'http://localhost:9200/logs-*/_search?q=event' | jq '.hits.total'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 6 — Retention policy (ILM concept)

```
curl -s -X PUT http://localhost:9200/_ilm/policy/logs-retention \
  -H 'Content-Type: application/json' \
  -d '{
    "policy": {
      "phases": {
        "hot":   { "actions": {} },
        "delete": { "min_age": "7d", "actions": { "delete": {} } }
      }
    }
  }' | jq
```

This enforces **7-day retention** — a CV0-004 logging sub-objective.

Step 7 — Aggregation query

```
curl -s -X POST http://localhost:9200/logs-*/_search \
  -H 'Content-Type: application/json' \
  -d '{"size":0,"aggs":{"by_severity":{"terms":{"field":"source.keyword"}}}}'
| jq
```

Step 8 — Cleanup

```
docker rm -f kibana logstash es
```

OpenSearch alternative (lighter)

If ELK is too heavy:

```
docker run -d --name os -p 9200:9200 -p 9600:9600 \
  -e discovery.type=single-node \
  -e DISABLE_SECURITY_PLUGIN=true \
  opensearchproject/opensearch:latest
```

Same API. Same Kibana clone is OpenSearch Dashboards.

Test it

Run these checks to prove the lab worked before you move on:

```
curl -s http://localhost:9200/_cluster/health | jq '.status'
curl -s 'http://localhost:9200/_cat/indices?v'
curl -s 'http://localhost:9200/logs-*/_search?q=event' | jq
'.hits.total.value'
curl -s http://localhost:5601/api/status | head -c 200
curl -s http://localhost:9200/_ilm/policy/logs-retention | jq '.["logs-
retention"].policy.phases.delete.min_age'
```

Expected: Run this before Step 8. Elasticsearch's cluster status is `"green"` or `"yellow"` (yellow is normal for single-node); `_cat/indices` lists a `logs-YYYY.MM.DD` index; the search reports a hit total of **20** — the 20 syslog lines Logstash ingested; Kibana's status shows `"level":"available"`; and the ILM policy returns `"7d"`, confirming the 7-day retention rule.

What you learned

- Centralized logging pipeline: collect → ship → store → query.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Retention is enforced with ILM policies.
- ELK and OpenSearch share the same query DSL.

Free tools used

- Elasticsearch / Kibana / Logstash — https://www.elastic.co/elastic-stack
- OpenSearch — https://opensearch.org
- rsyslog — https://www.rsyslog.com

Files in this lab

File	Purpose
`pipeline.conf`	Step 4 Logstash pipeline — TCP syslog input, mutate filter, Elasticsearch output.

Lab 18 — Distributed Tracing with Jaeger

Goal: Run Jaeger, instrument a Python 'checkout' path with OpenTelemetry and view a distributed trace of parent/child spans.

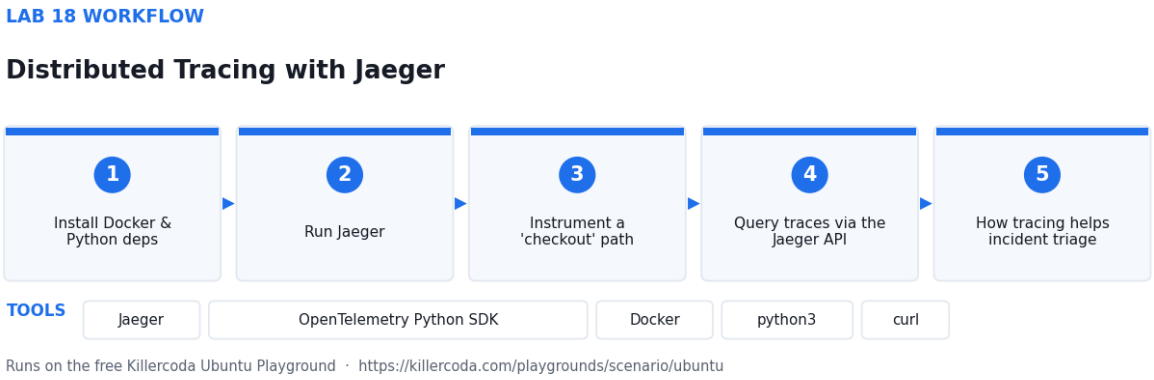

Figure 18 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Jaeger, OpenTelemetry Python SDK, Docker, python3, curl

In this lab you will run Jaeger all-in-one, instrument a small Python script with OpenTelemetry, and view a distributed trace in the Jaeger UI — covering the exam's tracing sub-objective.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Docker, Python deps

```
apt update && apt install -y docker.io python3-pip curl
systemctl start docker
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
pip3 install opentelemetry-api opentelemetry-sdk opentelemetry-exporter-otlp-
proto-grpc requests
```

Step 2 — Run Jaeger

```
docker run -d --name jaeger --net=host \
  jaegertracing/all-in-one:latest
sleep 5
curl -sI http://localhost:16686 | head -1
```

UI: `http://<killercoda-host>:16686`.

Step 3 — Instrument a "checkout" microservice path

Ready-made file: `app.py` — you can download it instead of typing this block.

```
mkdir -p /tmp/trace && cd /tmp/trace
cat > app.py <<'EOF'
import time, random
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter

trace.set_tracer_provider(TracerProvider())
trace.get_tracer_provider().add_span_processor(
    BatchSpanProcessor(OTLPSpanExporter(endpoint="localhost:4317",
insecure=True))
)
tracer = trace.get_tracer("shop")

def db_lookup():
    with tracer.start_as_current_span("db.lookup"):
        time.sleep(random.uniform(0.05, 0.2))

def charge():
    with tracer.start_as_current_span("payment.charge"):
        time.sleep(random.uniform(0.1, 0.3))

def checkout():
    with tracer.start_as_current_span("http.checkout"):
        db_lookup()
        charge()

for _ in range(5):
    checkout()
    time.sleep(0.3)
print("traces sent")
EOF

python3 app.py
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 4 — Query traces via Jaeger API

```
sleep 5
curl -s "http://localhost:16686/api/traces?service=shop&limit=5" | head -c 400
```

Open the UI, choose **service = shop**, **operation = http.checkout**, click a trace — you will see the parent span with two child spans (`db.lookup`, `payment.charge`) and their durations.

Step 5 — How tracing helps incident triage

A trace shows the **critical path** of a single request across all services. When a user reports "checkout is slow," you find the slowest span instead of guessing which service is at fault — this is why tracing is the third pillar of observability.

Symptom	Pillar that solves it
"CPU is high on host X"	Metrics (Lab 16)
"We had a 500 error at 10:42"	Logs (Lab 17)
"Why is *this specific request* slow?"	Traces (Lab 18)

Step 6 — Cleanup

```
docker rm -f jaeger
```

Test it

Run these checks to prove the lab worked before you move on:

```
curl -sI http://localhost:16686 | head -1
curl -s "http://localhost:16686/api/services" | jq '.data'
curl -s "http://localhost:16686/api/traces?service=shop&limit=5" | jq '.data |
length'
curl -s "http://localhost:16686/api/traces?service=shop&limit=1" | jq
'[.data[0].spans[].operationName]'
```

Expected: Run this before Step 6. Jaeger's UI returns `HTTP/1.1 200 OK`, the services list includes `"shop"`, the trace query returns **5 traces** (one per `checkout()` iteration), and each trace's span list contains `http.checkout` with its two children `db.lookup` and `payment.charge`.

What you learned

- Spans are the unit of tracing; traces are causally-linked spans.
- OpenTelemetry is the vendor-neutral instrumentation API.
- Tracing answers "where did this one request spend its time?".

Free tools used

- Jaeger — https://www.jaegertracing.io
- OpenTelemetry — https://opentelemetry.io
- Zipkin (alternative) — https://zipkin.io

Files in this lab

File	Purpose
`app.py`	Step 3 OpenTelemetry-instrumented checkout

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	script that emits parent and child spans to Jaeger.

Lab 19 — Horizontal Auto-Scaling Simulation

Goal: Simulate triggered, scheduled and manual horizontal scaling using a Bash controller that watches CPU and adds/removes Docker replicas.

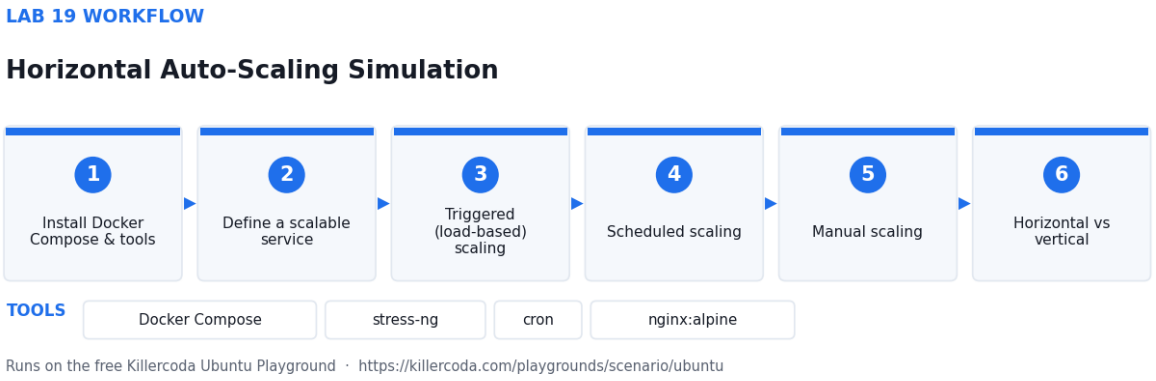

Figure 19 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Docker Compose, stress-ng, cron, nginx:alpine

In this lab you will simulate triggered, scheduled, and manual scaling of a service with Docker Compose plus a Bash controller that watches CPU and adds/removes replicas — the same pattern every cloud auto-scaler uses.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Docker Compose and load tools

```
apt update && apt install -y docker.io docker-compose-v2 stress-ng bc curl
systemctl start docker
```

Step 2 — Define a scalable service

Ready-made file: `docker-compose.yml` — you can download it instead of typing this block.

```
mkdir -p /tmp/scale && cd /tmp/scale
cat > docker-compose.yml <<'EOF'
services:
  app:
    image: nginx:alpine
    deploy: { replicas: 2 }
EOF
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker compose up -d --scale app=2
docker compose ps
```

Step 3 — Triggered scaling (load-based)

This loop reads host CPU every 5 s and scales between 2 and 6 replicas.

Ready-made file: `autoscale.sh` — you can download it instead of typing this block.

```
cat > /tmp/scale/autoscale.sh <<'EOF'
#!/bin/bash
cd /tmp/scale
MIN=2; MAX=6
while true; do
  CPU=$(awk -v cores=$(nproc) '/cpu / {usage=($2+$4)*100/($2+$4+$5)} END{print
usage}' /proc/stat)
  CUR=$(docker compose ps -q app | wc -l)
  TARGET=$CUR
  awk -v c="$CPU" 'BEGIN{exit !(c+0 > 70)}' && [ $CUR -lt $MAX ] && TARGET=$
((CUR+1))
  awk -v c="$CPU" 'BEGIN{exit !(c+0 < 20)}' && [ $CUR -gt $MIN ] && TARGET=$
((CUR-1))
  if [ "$TARGET" != "$CUR" ]; then
    echo "$(date +%T) CPU=${CPU}%  scaling $CUR -> $TARGET"
    docker compose up -d --scale app=$TARGET --no-recreate
  fi
  sleep 5
done
EOF
chmod +x /tmp/scale/autoscale.sh
/tmp/scale/autoscale.sh &
ASPID=$!
```

Generate load:

```
stress-ng --cpu 4 --timeout 40s
sleep 60
```

You will see the controller scale **up** during stress and **down** afterwards. This is the **trending/load/event** trigger family from the exam objectives.

Step 4 — Scheduled scaling

```
echo "*/1 * * * * cd /tmp/scale && docker compose up -d --scale app=4 --no-
recreate" | crontab -
crontab -l
```

A cron-driven pre-scale is the "scheduled" approach — useful before a known traffic peak.

Step 5 — Manual scaling

```
docker compose up -d --scale app=3 --no-recreate
docker compose ps
```

Step 6 — Horizontal vs vertical

Type	Action	Example

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Horizontal	Add/remove replicas	This lab; AWS ASG, K8s HPA
Vertical	Resize a single instance	Stop/start with bigger CPU/RAM

Horizontal is the cloud default because it survives instance loss and scales linearly.

Step 7 — Cleanup

```
kill $ASPID 2>/dev/null
crontab -r 2>/dev/null
docker compose down
```

Test it

Run these checks to prove the lab worked before you move on:

```
cd /tmp/scale && docker compose ps
docker compose ps -q app | wc -l
crontab -l
docker stats --no-stream --format '{{.Name}}\t{{.CPUPerc}}'
```

Expected: Run this before Step 7. `docker compose ps` shows every `app` replica **running**, the replica count sits between the controller's MIN of 2 and MAX of 6 (it rises toward 6 while `stress-ng` runs and falls back to 2 afterwards), `crontab -l` shows the scheduled `--scale app=4` entry, and `docker stats` reports live CPU for each replica.

What you learned

- Triggered, scheduled, and manual scaling.
- A scaler is just a control loop watching a metric.
- Horizontal scaling is the cloud-native default.

Free tools used

- Docker Compose — https://docs.docker.com/compose
- stress-ng — https://github.com/ColinIanKing/stress-ng
- cron (built-in)

Files in this lab

File	Purpose
`docker-compose.yml`	Step 2 Compose stack — the scalable `app` service.
`autoscale.sh`	Step 3 auto-scaling control loop — watches host CPU and scales between 2 and 6 replicas.

Lab 20 — Backup & Recovery with restic

Goal: Perform encrypted full/incremental/differential backups to off-site S3 storage, then test recoverability and verify integrity.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Figure 20 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: restic, MinIO, Docker

In this lab you will use **restic** — a free, open-source, encrypted, deduplicating backup tool — to perform **full**, **incremental**, and **differential**-style snapshots, store them off-site (MinIO/S3), test recoverability, and verify integrity.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install restic and MinIO (off-site target)

```
apt update && apt install -y restic docker.io
systemctl start docker

docker run -d --name minio -p 9000:9000 \
  -e MINIO_ROOT_USER=admin -e MINIO_ROOT_PASSWORD=cloudplus \
  minio/minio server /data
sleep 4
docker exec minio mc alias set local http://127.0.0.1:9000 admin cloudplus
docker exec minio mc mb local/backups
```

Step 2 — Configure restic to talk to S3-compatible storage

```
export RESTIC_REPOSITORY="s3:http://localhost:9000/backups"
export AWS_ACCESS_KEY_ID=admin
export AWS_SECRET_ACCESS_KEY=cloudplus
export RESTIC_PASSWORD=cloudplus

restic init
```

The repo is **encrypted at rest** with `RESTIC_PASSWORD`.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 3 — Create source data and take a FULL backup

```
mkdir -p /data/app && for i in 1 2 3; do
  dd if=/dev/urandom of=/data/app/file$i bs=1M count=2 2>/dev/null
done
restic backup /data/app --tag full
restic snapshots
```

Step 4 — INCREMENTAL backup (only changed blocks)

```
echo "new content" >> /data/app/file1
dd if=/dev/urandom of=/data/app/file4 bs=1M count=1 2>/dev/null
restic backup /data/app --tag incremental
restic stats latest
```

restic deduplicates at block level — only deltas travel.

Step 5 — Differential-style: snapshot since the full

```
restic backup /data/app --tag diff --parent $(restic snapshots --tag full --
json | python3 -c 'import json,sys; print(json.load(sys.stdin)[0]
["short_id"])')
restic snapshots
```

Step 6 — Recoverability test (granular restore)

```
mkdir -p /restore
SNAP=$(restic snapshots --json | python3 -c 'import json,sys;
print(json.load(sys.stdin)[-1]["short_id"])')
restic restore $SNAP --target /restore --include /data/app/file1
ls -lh /restore/data/app/
```

Step 7 — Bulk restore (whole snapshot)

```
rm -rf /data/app
restic restore latest --target /
ls -lh /data/app/
```

Step 8 — Integrity check

```
restic check --read-data-subset=10%
```

`check` re-reads a subset and verifies hashes — the **integrity** sub-objective.

Step 9 — Retention policy

```
restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 12 --prune
```

Step 10 — Backup matrix

Type	What it stores	Restore speed
Full	Everything	Fastest (one snapshot)
Incremental	Changes since last backup of any kind	Slower (chain)
Differential	Changes since last full	Two snapshots

restic technically does **content-defined chunking** — every backup is a full restorable point but only sends deltas.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 11 — Cleanup

```
docker rm -f minio
rm -rf /data/app /restore
```

Test it

Run these checks to prove the lab worked before you move on:

```
restic snapshots
restic stats latest
restic check --read-data-subset=10%
ls -lh /data/app/
docker exec minio mc ls local/backups/
```

Expected: Run this before Step 11. `restic snapshots` lists three snapshots tagged `full`, `incremental` and `diff`; `restic stats` reports the restored size of the latest snapshot; `restic check` ends with **no errors were found**; `/data/app/` contains `file1`–`file4` again after the Step 7 bulk restore; and the MinIO `backups` bucket holds the restic repository objects (`config`, `data/`, `snapshots/`).

What you learned

- Encrypted, deduplicated backups to S3-compatible storage.
- Full / incremental / differential semantics.
- Recoverability and integrity tests are mandatory — not optional.

Free tools used

- restic — https://restic.net
- MinIO — https://min.io
- Borg (alternative) — https://www.borgbackup.org
- Duplicity (alternative) — https://duplicity.gitlab.io

Lab 21 — Patching & Lifecycle Management

Goal: Simulate a cloud resource lifecycle from provision through minor patch, major upgrade, testing and clean decommissioning.

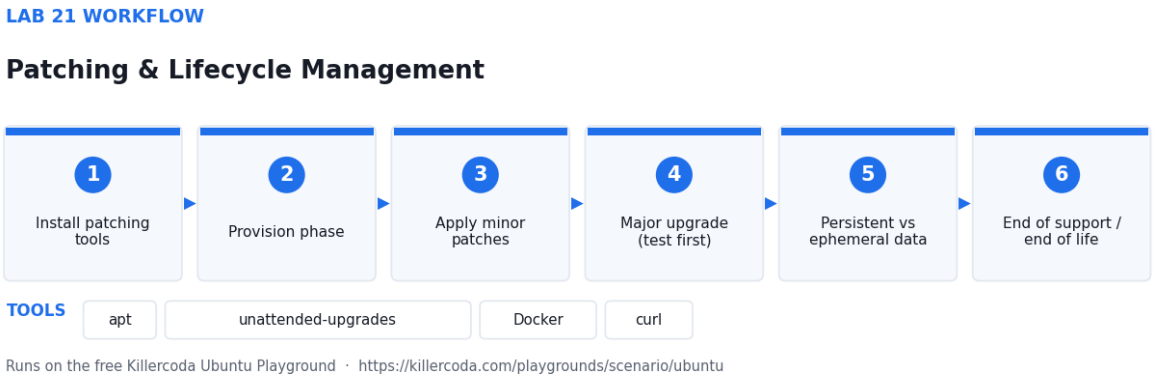

Figure 21 — the workflow you build in this lab.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: apt, unattended-upgrades, Docker, curl

In this lab you will simulate the lifecycle of a cloud resource: **provision → patch (minor) → upgrade (major) → test → decommission**. You will use `apt`, `unattended-upgrades`, container image tags, and clean teardown.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install patching tools

```
apt update && apt install -y unattended-upgrades apt-listchanges docker.io
systemctl start docker
```

Step 2 — Provision phase

```
docker run -d --name web --label lifecycle=active nginx:1.24-alpine
docker inspect -f '{{.Config.Image}}' web
```

Step 3 — Apply MINOR patches (no behaviour change)

```
apt-get -s upgrade | head -20             # simulate
unattended-upgrade --dry-run -d 2>&1 | tail -10
```

`unattended-upgrades` automates security minors. The CV0-004 distinction:

- **Minor** = bug fixes, security patches (1.24.0 → 1.24.5)
- **Major** = breaking changes (1.24 → 1.25)

Step 4 — MAJOR upgrade (test first!)

Stage the new version side-by-side:

```
docker run -d --name web-new nginx:1.27-alpine
docker exec web-new nginx -v
```

Run a smoke test:

```
curl -sI http://$(docker inspect -f '{{(index .NetworkSettings.Networks
"bridge").IPAddress}}' web-new)/ | head -1
```

If it passes, swap (a Lab-10 style blue-green):

```
docker rm -f web
docker rename web-new web
```

Step 5 — Persistent vs ephemeral data during upgrade

Persistent data (volumes, databases) **must survive** the swap. Ephemeral state (cache, /tmp) is expected to vanish.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker volume create app-data
docker run -d --name app2 -v app-data:/var/lib/app nginx:alpine
docker exec app2 sh -c 'echo persistent > /var/lib/app/keep.txt'
docker rm -f app2
docker run --rm -v app-data:/var/lib/app alpine cat /var/lib/app/keep.txt
```

Step 6 — End of support / End of life

```
docker pull alpine:3.10 2>&1 | head -3   # old release
docker run --rm alpine:3.10 cat /etc/alpine-release
```

Alpine 3.10 hit **end of support** in 2021. Running EoS images means **no security patches** — a cloud-governance violation in most orgs.

Step 7 — Decommissioning

Tag, snapshot, then destroy.

```
docker commit web web-decom-$(date +%Y%m%d)
docker images | grep decom
docker rm -f web
docker volume rm app-data
docker rmi nginx:1.24-alpine 2>/dev/null
```

The **commit** is your archival snapshot; the rm/rmi is the decommission.

Step 8 — Lifecycle phases summary

1. Provision (Lab 13/14 — IaC/CaC)
2. Configure (Lab 14)
3. Monitor (Lab 16-18)
4. Patch (this lab)
5. Scale (Lab 19)
6. Backup (Lab 20)
7. Decommission (this lab)

Test it

Run these checks to prove the lab worked before you move on:

```
docker inspect -f '{{.Config.Image}}' web
docker exec web nginx -v
docker images | grep decom
docker volume ls | grep app-data
docker run --rm -v app-data:/var/lib/app alpine cat /var/lib/app/keep.txt
```

Expected: Run this before Step 7's `docker rm`/`volume rm`. The running `web` container now reports image `nginx:1.27-alpine` (the major upgrade replaced 1.24 and kept the name), `nginx -v` prints `nginx version: nginx/1.27.x`, the archival `web-decom-<date>` image is listed, the `app-data` volume still exists, and reading it prints `persistent` — the persistent data survived the container swap.

What you learned

- Minor vs major upgrades have different risk profiles.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Persistent data must be preserved across upgrades.
- EoL/EoS items must be replaced — not just patched.
- Decommissioning includes archive + revoke + delete.

Free tools used

- apt / unattended-upgrades (built-in)
- Docker — https://www.docker.com
- Pkgs.org (find current versions) — https://pkgs.org
- endoflife.date — https://endoflife.date

Domain 4 — Security (19%)

Securing the cloud: vulnerability management, CIS hardening, IAM & RBAC, secure access (bastion + MFA), secrets management, web application firewalls and runtime threat detection.

What this domain covers in the labs:

- **Lab 22 — Vulnerability Scanning with Trivy:** Scan a container image, filesystem and IaC config with Trivy following the scope → identify → assess → remediate workflow.
- **Lab 23 — CIS Benchmark Audit with Lynis:** Audit the VM against the CIS Ubuntu Benchmark with Lynis, score it, remediate findings and re-audit to confirm improvement.
- **Lab 24 — IAM & RBAC with Keycloak:** Stand up Keycloak and build a full IAM stack — realm, roles, users, an OAuth2 client — then authenticate and inspect RBAC roles in the JWT.
- **Lab 25 — Bastion Host with SSH Key + MFA:** Configure key-based SSH with TOTP MFA and route access through a locked-down bastion jump host — the canonical secure cloud-VM access pattern.
- **Lab 26 — Secrets Management with HashiCorp Vault:** Run Vault to store static secrets, mint auto-expiring dynamic DB credentials and audit every access.
- **Lab 27 — Web Application Firewall with ModSecurity:** Deploy Nginx + ModSecurity + the OWASP Core Rule Set and watch it block SQLi, XSS and path-traversal, then tune false positives.
- **Lab 28 — Detecting Suspicious Activity with Falco:** Run Falco and trigger runtime detections that map to CV0-004 attack categories, then route alerts to Prometheus / SIEM.

Lab 22 — Vulnerability Scanning with Trivy

Goal: Scan a container image, filesystem and IaC config with Trivy following the scope → identify → assess → remediate workflow.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Figure 22 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Trivy, Docker, sample Terraform

In this lab you will run **Trivy** against a container image, the local filesystem, and an IaC config. You will follow the exam's vulnerability-management steps: **scope** → **identify** → **assess** → **remediate**.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Trivy

```
apt update && apt install -y wget docker.io
systemctl start docker
wget -qO- https://aquasecurity.github.io/trivy-repo/deb/public.key | apt-key
add -
echo "deb https://aquasecurity.github.io/trivy-repo/deb generic main" >
/etc/apt/sources.list.d/trivy.list
apt update && apt install -y trivy
trivy --version
```

Step 2 — Scoping

Pick what to scan. For this lab:

1. A known-vulnerable image (`vulnerables/web-dvwa`)
2. The Killercoda VM filesystem
3. A Terraform file from Lab 13

Step 3 — Identify (image scan)

```
trivy image --severity HIGH,CRITICAL alpine:3.10 | head -50
```

Each finding includes a **CVE** ID, package, fixed version, and CVSS score.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 4 — Assess

CVSS = severity × exploitability. The exam expects you to know CVE/CVSS terms.

```
trivy image --severity CRITICAL --format json alpine:3.10 | head -c 600
```

Group by severity:

```
trivy image alpine:3.10 --format table | tail -5
```

Step 5 — Remediate (rebuild with patched base)

```
trivy image alpine:3.20 --severity CRITICAL | tail -10
```

`alpine:3.20` ships with patched packages. Remediation is usually:

1. Bump the base image.
2. Rebuild the application image.
3. Push and redeploy.

Step 6 — Filesystem scan

```
trivy fs --security-checks vuln,secret /etc | head -30
```

`secret` detection finds API keys / private keys committed by mistake.

Step 7 — IaC scan (misconfiguration)

Ready-made file: `main.tf` — you can download it instead of typing this block.

```
mkdir -p /tmp/tf-bad && cat > /tmp/tf-bad/main.tf <<'EOF'
resource "aws_s3_bucket" "bad" { bucket = "world-readable" }
resource "aws_s3_bucket_acl" "bad" { bucket = aws_s3_bucket.bad.id  acl =
"public-read" }
EOF

trivy config /tmp/tf-bad
```

Trivy flags the public-read S3 bucket — an **outdated/insecure component definition**.

Step 8 — Daily automation hint

Add to a CI pipeline (Lab 30):

```
trivy image --exit-code 1 --severity CRITICAL myorg/myapp:latest
```

Non-zero exit fails the build — this is the "gate" that stops vulnerable images from shipping.

Test it

Run these checks to prove the lab worked before you move on:

```
trivy --version
trivy image --severity HIGH,CRITICAL alpine:3.10 | tail -20
trivy image --severity CRITICAL alpine:3.20 | tail -10
trivy config /tmp/tf-bad
trivy fs --security-checks vuln,secret /etc | head -20
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: Trivy prints its version and a vulnerability DB date; the `alpine:3.10` scan lists **multiple HIGH/CRITICAL CVEs** with CVE IDs, installed and fixed versions; the same scan of `alpine:3.20` reports **no CRITICAL vulnerabilities** (`Total: 0`), showing that bumping the base image is the remediation; and `trivy config` flags the `/tmp/tf-bad/main.tf` public-read S3 bucket as a HIGH misconfiguration.

What you learned

- Vulnerability scanning closes the **identify** step.
- CVE + CVSS is the universal vocabulary.
- Image, filesystem, and IaC scans are all part of cloud security hygiene.

Free tools used

- Trivy — https://aquasecurity.github.io/trivy
- Grype (alternative) — https://github.com/anchore/grype
- OpenVAS / Greenbone CE — https://www.greenbone.net
- NVD CVE database — https://nvd.nist.gov
- CVE.org — https://www.cve.org

Files in this lab

File	Purpose
`main.tf`	Step 7 deliberately insecure Terraform — the public-read S3 bucket Trivy flags.

Lab 23 — CIS Benchmark Audit with Lynis

Goal: Audit the VM against the CIS Ubuntu Benchmark with Lynis, score it, remediate findings and re-audit to confirm improvement.

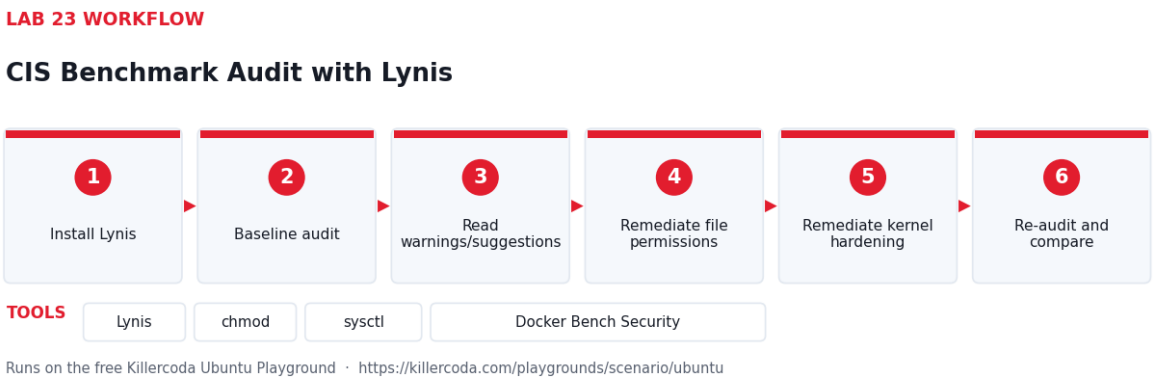

Figure 23 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Lynis, chmod, sysctl, Docker Bench Security

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

In this lab you will audit the Killercoda VM against the **CIS Ubuntu Benchmark** using **Lynis**, score it, and remediate one finding — covering the exam's *benchmark, hardening, patching* sub-objectives.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Lynis

```
apt update && apt install -y lynis
lynis --version
```

Step 2 — Run a baseline audit

```
lynis audit system --quiet
```

When the run finishes, look for:

```
Hardening index : 60 [#######        ]
```

The **hardening index** is your CIS score. Tertiary Infotech treats anything below 80 as a fail.

Step 3 — Read the warnings

```
grep -E '^\s+\!' /var/log/lynis.log | head -20
lynis show warnings
lynis show suggestions | head -20
```

Each suggestion maps to a CIS control number — for example *file permissions on /etc/shadow*, *unused kernel modules*, *no MOTD banner*.

Step 4 — Remediate one finding (set permissions)

```
chmod 600 /etc/shadow /etc/gshadow
chmod 644 /etc/passwd /etc/group
ls -l /etc/shadow /etc/passwd
```

Step 5 — Remediate a kernel hardening finding

> *Ready-made file: `99-hardening.conf` — you can download it instead of typing this block.*

```
cat > /etc/sysctl.d/99-hardening.conf <<'EOF'
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.tcp_syncookies = 1
kernel.randomize_va_space = 2
EOF
sysctl --system | tail -10
```

Step 6 — Re-audit and compare

```
lynis audit system --quiet
grep "Hardening index" /var/log/lynis.log
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The score should rise.

Step 7 — Vendor-specific benchmarks

Lynis covers Ubuntu/RHEL OS hardening. For cloud-vendor benchmarks use:

- **Prowler** (AWS, Azure, GCP, K8s) — https://github.com/prowler-cloud/prowler
- **kube-bench** (CIS Kubernetes) — https://github.com/aquasecurity/kube-bench
- **Docker Bench** (CIS Docker) — https://github.com/docker/docker-bench-security

Quick demo:

```
docker run --rm --net host --pid host --userns host --cap-add audit_control \
  -v /etc:/etc:ro -v /var/lib:/var/lib:ro -v
/var/run/docker.sock:/var/run/docker.sock:ro \
  --label docker_bench_security \
  docker/docker-bench-security 2>&1 | head -40
```

Step 8 — Cleanup

```
rm -f /etc/sysctl.d/99-hardening.conf
sysctl --system >/dev/null
```

Test it

Run these checks to prove the lab worked before you move on:

```
grep "Hardening index" /var/log/lynis.log
lynis show warnings
lynis show suggestions | head -20
ls -l /etc/shadow /etc/passwd
sysctl net.ipv4.tcp_syncookies kernel.randomize_va_space
net.ipv4.conf.all.send_redirects
```

Expected: Run this before Step 8. The log shows a `Hardening index : NN [######  ]` line that is **higher after the Step 6 re-audit than the Step 2 baseline**; `/etc/shadow` is `-rw-------` (600) and `/etc/passwd` is `-rw-r--r--` (644); and the kernel now reports `net.ipv4.tcp_syncookies = 1`, `kernel.randomize_va_space = 2` and `net.ipv4.conf.all.send_redirects = 0` from the hardening sysctl file.

What you learned

- Benchmarks are objective targets you can measure against.
- Hardening = closing the gap between current and benchmark.
- Different benchmarks for OS, Docker, K8s, AWS, Azure.

Free tools used

- Lynis — https://cisofy.com/lynis
- CIS Benchmarks (free PDFs) — https://www.cisecurity.org/cis-benchmarks
- Prowler — https://github.com/prowler-cloud/prowler
- kube-bench — https://github.com/aquasecurity/kube-bench
- Docker Bench Security — https://github.com/docker/docker-bench-security

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Files in this lab

File	Purpose
`99-hardening.conf`	Step 5 sysctl hardening file that raises the Lynis hardening index.

Lab 24 — IAM & RBAC with Keycloak

Goal: Stand up Keycloak and build a full IAM stack — realm, roles, users, an OAuth2 client — then authenticate and inspect RBAC roles in the JWT.

Figure 24 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Keycloak, Docker, curl, jq

In this lab you will run **Keycloak** (open-source identity provider), create a realm, users, roles, and an OAuth 2.0 client — the building blocks of cloud IAM (federation, SAML, OIDC, RBAC).

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Run Keycloak

```
apt update && apt install -y docker.io curl jq
systemctl start docker

docker run -d --name kc -p 8080:8080 \
  -e KEYCLOAK_ADMIN=admin \
  -e KEYCLOAK_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:25.0 start-dev
sleep 25
curl -sI http://localhost:8080/realms/master | head -1
```

UI: `http://<killercoda-host>:8080`.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 2 — Get an admin token

```
TOKEN=$(curl -s -X POST http://localhost:8080/realms/master/protocol/openid-
connect/token \
  -d "username=admin" -d "password=admin" -d "grant_type=password" -d
"client_id=admin-cli" \
  | jq -r .access_token)
echo "$TOKEN" | head -c 40
```

This is an **OIDC** access token. Cloud admin CLIs do exactly the same call against AWS Cognito / Azure AD / GCP Identity.

Step 3 — Create a realm (= a tenant)

```
curl -s -X POST http://localhost:8080/admin/realms \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"realm":"cloudplus","enabled":true}'
```

Step 4 — Create roles (RBAC)

```
for role in viewer operator admin; do
  curl -s -X POST http://localhost:8080/admin/realms/cloudplus/roles \
    -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
    -d "{\"name\":\"$role\"}"
done

curl -s http://localhost:8080/admin/realms/cloudplus/roles \
  -H "Authorization: Bearer $TOKEN" | jq '.[].name'
```

Step 5 — Create a user and assign a role

```
curl -s -X POST http://localhost:8080/admin/realms/cloudplus/users \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"username":"alice","enabled":true,"credentials":
[{"type":"password","value":"cloud","temporary":false}]}'

UID=$(curl -s "http://localhost:8080/admin/realms/cloudplus/users?
username=alice" \
  -H "Authorization: Bearer $TOKEN" | jq -r '.[0].id')

ROLE=$(curl -s http://localhost:8080/admin/realms/cloudplus/roles/operator \
  -H "Authorization: Bearer $TOKEN")

curl -s -X POST http://localhost:8080/admin/realms/cloudplus/users/$UID/role-
mappings/realm \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d "[$ROLE]"
```

Step 6 — Create an OAuth 2.0 client

```
curl -s -X POST http://localhost:8080/admin/realms/cloudplus/clients \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"clientId":"cli-
app","publicClient":true,"directAccessGrantsEnabled":true,"redirectUris":
["*"]}'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 7 — Authenticate as Alice (token-based)

```
ALICE_TOKEN=$(curl -s -X POST
http://localhost:8080/realms/cloudplus/protocol/openid-connect/token \
  -d "username=alice" -d "password=cloud" -d "grant_type=password" -d
"client_id=cli-app" \
  | jq -r .access_token)

# Decode the JWT payload
echo "$ALICE_TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq .realm_access
```

The payload includes `realm_access.roles: ["operator"]` — that is **role-based access control**.

Step 8 — Authentication models map

Exam term	Where you saw it
Local users	`username/password` in Keycloak
Federation / SAML	Keycloak → external IdP
OpenID Connect / OIDC	the `/protocol/openid-connect/*` endpoints
Token-based	bearer JWT
Directory-based	LDAP user federation in Keycloak
MFA	OTP setting per user (Lab 25)

Step 9 — Cleanup

```
docker rm -f kc
```

Test it

Run these checks to prove the lab worked before you move on:

```
curl -sI http://localhost:8080/realms/master | head -1
curl -s http://localhost:8080/admin/realms -H "Authorization: Bearer $TOKEN" |
jq '.[].realm'
curl -s http://localhost:8080/admin/realms/cloudplus/roles -H "Authorization:
Bearer $TOKEN" | jq '.[].name'
curl -s "http://localhost:8080/admin/realms/cloudplus/users?username=alice" -H
"Authorization: Bearer $TOKEN" | jq '.[].username'
echo "$ALICE_TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq .realm_access
```

Expected: Run this before Step 9. Keycloak answers `HTTP/1.1 200 OK` on the master realm; the realm list includes `"cloudplus"` next to `"master"`; the role list contains `viewer`, `operator` and `admin`; the user query returns `"alice"`; and Alice's decoded JWT payload shows `"roles"` containing **`"operator"`** — the RBAC assignment travelled in the token claim.

What you learned

- Identity provider, realms, users, roles, clients.
- OAuth 2.0 + OIDC token flow.
- RBAC roles propagate via JWT claims.

Free tools used

- Keycloak — https://www.keycloak.org

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- jwt.io decoder (web) — https://jwt.io
- OAuth.tools — https://oauth.tools

Lab 25 — Bastion Host with SSH Key + MFA

Goal: Configure key-based SSH with TOTP MFA and route access through a locked-down bastion jump host — the canonical secure cloud-VM access pattern.

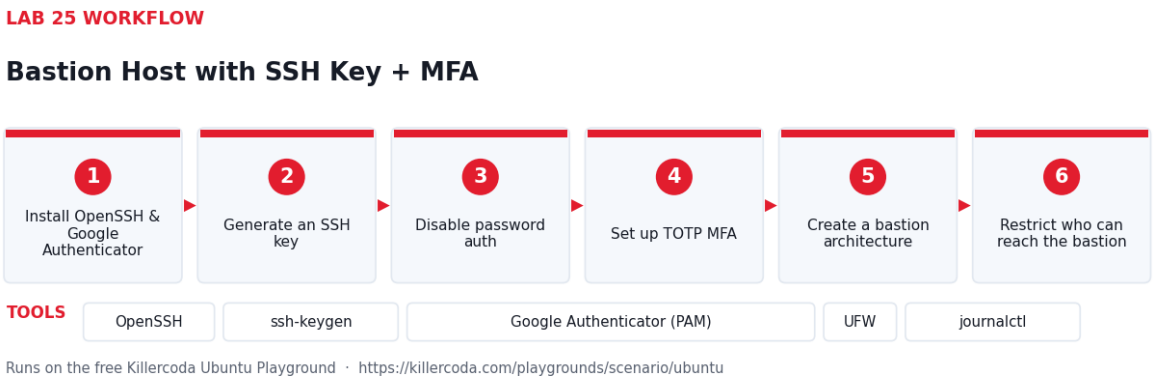

Figure 25 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: OpenSSH, ssh-keygen, Google Authenticator (PAM), UFW, journalctl

In this lab you will configure SSH key-based authentication, add **Google Authenticator TOTP** as a second factor, and route through a **bastion** (jump host) — the canonical secure-access pattern for cloud VMs.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install OpenSSH server and Google Authenticator

```
apt update && apt install -y openssh-server libpam-google-authenticator
systemctl start ssh
ss -ltn | grep :22
```

Step 2 — Generate SSH key (no password)

```
ssh-keygen -t ed25519 -N "" -f /root/.ssh/id_ed25519
cat /root/.ssh/id_ed25519.pub >> /root/.ssh/authorized_keys
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys
```

Test:

```
ssh -o StrictHostKeyChecking=no localhost "echo SSH key auth OK"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 3 — Disable password auth, allow only keys

```
sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication no/'
/etc/ssh/sshd_config
sed -i 's/^#*PubkeyAuthentication.*/PubkeyAuthentication yes/'
/etc/ssh/sshd_config
sed -i 's/^#*ChallengeResponseAuthentication.*/ChallengeResponseAuthentication
yes/' /etc/ssh/sshd_config
sed -i 's/^#*UsePAM.*/UsePAM yes/' /etc/ssh/sshd_config
echo 'AuthenticationMethods publickey,keyboard-interactive' >>
/etc/ssh/sshd_config
```

Step 4 — Set up TOTP MFA for root

```
google-authenticator -t -d -f -r 3 -R 30 -W -q -s /root/.google_authenticator
ls -l /root/.google_authenticator
head -1 /root/.google_authenticator
```

The first line is the **TOTP seed** — scan that as a QR with Google Authenticator / Authy / FreeOTP on your phone.

Wire it into PAM:

```
sed -i '1a auth required pam_google_authenticator.so' /etc/pam.d/sshd
systemctl restart ssh
```

Now SSH requires **key + 6-digit TOTP**.

Step 5 — Create a "bastion" architecture

Run a private app behind a network namespace; the bastion is the only way in.

```
docker run -d --name app nginx:alpine
APP_IP=$(docker inspect -f '{{(index .NetworkSettings.Networks
"bridge").IPAddress}}' app)
echo "App IP (private): $APP_IP"

# Bastion = your Killercoda root SSH on port 22
# Client jump:
ssh -J root@localhost root@$APP_IP "echo 'reached private app via bastion' "
2>&1 || true
```

The pattern in production: **internet → bastion (only host with public IP) → private subnet hosts**.

Step 6 — Restrict who can reach the bastion

```
ufw enable >/dev/null 2>&1 || apt install -y ufw
ufw allow from 10.0.0.0/8 to any port 22 proto tcp
ufw status verbose
```

Lock the bastion to corporate CIDRs only.

Step 7 — Audit trail

```
journalctl -u ssh -n 20 --no-pager
last -n 5
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

`last`/`journalctl` give you the **audit trail** required by SOC2 / ISO 27001.

Step 8 — Cleanup

```
docker rm -f app
sed -i '/AuthenticationMethods/d' /etc/ssh/sshd_config
sed -i '/pam_google_authenticator/d' /etc/pam.d/sshd
systemctl restart ssh
ufw disable 2>/dev/null
```

Test it

Run these checks to prove the lab worked before you move on:

```
ss -ltn | grep :22
ssh -o StrictHostKeyChecking=no localhost "echo SSH key auth OK"
grep -E '^(PasswordAuthentication|PubkeyAuthentication|AuthenticationMethods)'
/etc/ssh/sshd_config
ls -l /root/.google_authenticator
grep pam_google_authenticator /etc/pam.d/sshd
ufw status verbose
```

Expected: Run this before Step 8. `ss` shows sshd LISTENing on port 22; the key-based SSH prints `SSH key auth OK` with no password prompt; the sshd config reads `PasswordAuthentication no`, `PubkeyAuthentication yes` and `AuthenticationMethods publickey,keyboard-interactive`; the `/root/.google_authenticator` TOTP seed file exists with `-r--------` permissions; PAM includes the `pam_google_authenticator.so` line; and `ufw status` shows port 22 allowed only from `10.0.0.0/8`.

What you learned

- SSH key auth, MFA via TOTP, and bastion jump-hosts.
- AuthenticationMethods chains factors.
- Audit trails come for free with systemd-journald.

Free tools used

- OpenSSH — https://www.openssh.com
- Google Authenticator PAM — https://github.com/google/google-authenticator-libpam
- FreeOTP (mobile, free) — https://freeotp.github.io
- Authy / Microsoft Authenticator — free mobile apps

Lab 26 — Secrets Management with HashiCorp Vault

Goal: Run Vault to store static secrets, mint auto-expiring dynamic DB credentials and audit every access.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

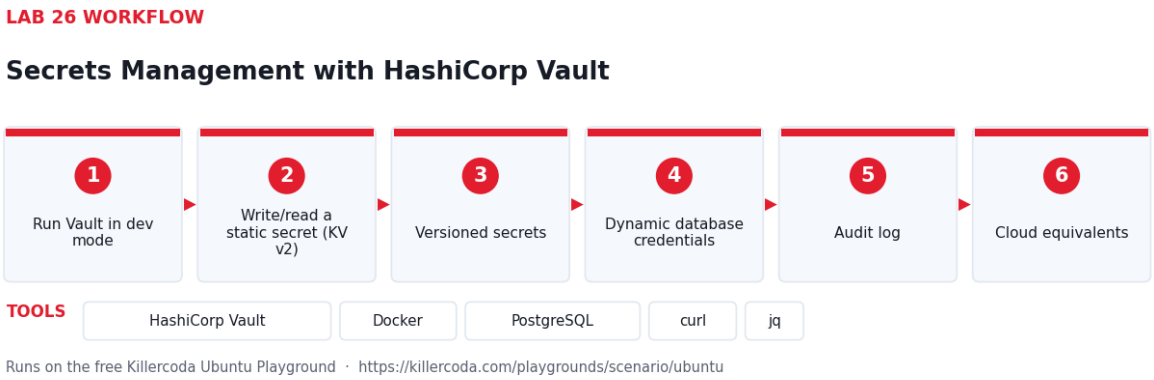

Figure 26 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: HashiCorp Vault, Docker, PostgreSQL, curl, jq

In this lab you will run **HashiCorp Vault** in dev mode, store secrets, generate dynamic database credentials, and rotate them — the secrets-management sub-objective of CV0-004.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Run Vault in dev mode

```
apt update && apt install -y docker.io curl jq
systemctl start docker

docker run -d --name vault \
  --cap-add IPC_LOCK -p 8200:8200 \
  -e 'VAULT_DEV_ROOT_TOKEN_ID=cloudplus' \
  -e 'VAULT_DEV_LISTEN_ADDRESS=0.0.0.0:8200' \
  hashicorp/vault:latest
sleep 4

export VAULT_ADDR=http://localhost:8200
export VAULT_TOKEN=cloudplus
docker exec vault vault status
```

Step 2 — Write and read a static secret (KV v2)

```
docker exec -e VAULT_TOKEN=cloudplus vault \
  vault kv put secret/app/db username=admin password=Sup3rSecret

docker exec -e VAULT_TOKEN=cloudplus vault \
  vault kv get secret/app/db
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Hard-coded secrets in code → ❌. Reference from Vault → ✅.

Step 3 — Versioned secrets (audit + rollback)

```
docker exec -e VAULT_TOKEN=cloudplus vault \
  vault kv put secret/app/db username=admin password=Rotat3d!

docker exec -e VAULT_TOKEN=cloudplus vault \
  vault kv get -version=1 secret/app/db
```

You just rolled back to v1. Versioning is built in.

Step 4 — Dynamic database credentials

Run a Postgres + tell Vault to mint **short-lived** users on demand.

```
docker run -d --name pg --network=host -e POSTGRES_PASSWORD=root postgres:16
sleep 5

docker exec -e VAULT_TOKEN=cloudplus vault sh -c '
  vault secrets enable database
  vault write database/config/pg \
    plugin_name=postgresql-database-plugin \
    allowed_roles="readonly" \
    connection_url="postgresql://{{username}}:{{password}}
@host.docker.internal:5432/postgres?sslmode=disable" \
    username="postgres" password="root"
  vault write database/roles/readonly \
    db_name=pg \
    creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD
\"{{password}}\" VALID UNTIL \"{{expiration}}\"; GRANT pg_read_all_data TO
\"{{name}}\";" \
    default_ttl="2m" max_ttl="5m"
'

docker exec -e VAULT_TOKEN=cloudplus vault vault read database/creds/readonly
```

You now have a **2-minute-lifetime** Postgres user. After 2 minutes Vault deletes it. Compromise risk → near zero.

Step 5 — Audit log (compliance)

```
docker exec -e VAULT_TOKEN=cloudplus vault \
  vault audit enable file file_path=/vault/logs/audit.log

docker exec -e VAULT_TOKEN=cloudplus vault \
  vault kv get secret/app/db >/dev/null

docker exec vault tail -3 /vault/logs/audit.log
```

Every secret access is logged. SOC2 / PCI-DSS love this.

Step 6 — Cloud equivalents

Tool	Equivalent service
Vault	AWS Secrets Manager, AWS SSM Parameter

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	Store, Azure Key Vault, GCP Secret Manager
Vault dynamic DB role	RDS IAM auth, Azure Managed Identity for SQL

Step 7 — Cleanup

```
docker rm -f vault pg
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker exec vault vault status
docker exec -e VAULT_TOKEN=cloudplus vault vault kv get secret/app/db
docker exec -e VAULT_TOKEN=cloudplus vault vault kv get -version=1
secret/app/db
docker exec -e VAULT_TOKEN=cloudplus vault vault read database/creds/readonly
docker exec vault tail -3 /vault/logs/audit.log
```

Expected: Run this before Step 7. `vault status` shows `Sealed false` and `Initialized true`; the current secret reads back at **version 2** with `password Rotat3d!` while `-version=1` still returns `Sup3rSecret` (versioned rollback works); `database/creds/readonly` mints a fresh username like `v-root-readonly-<random>` with `lease_duration 2m`; and the audit log tail shows JSON entries recording each secret read.

What you learned

- Static and dynamic secrets.
- Versioning, audit, lease/expiry.
- Dynamic creds collapse the blast radius of leaked passwords.

Free tools used

- HashiCorp Vault — https://www.vaultproject.io
- SOPS (alternative) — https://github.com/getsops/sops
- age (file encryption) — https://github.com/FiloSottile/age
- Mozilla SOPS + age + git — common GitOps secrets pattern

Lab 27 — Web Application Firewall with ModSecurity

Goal: Deploy Nginx + ModSecurity + the OWASP Core Rule Set and watch it block SQLi, XSS and path-traversal, then tune false positives.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

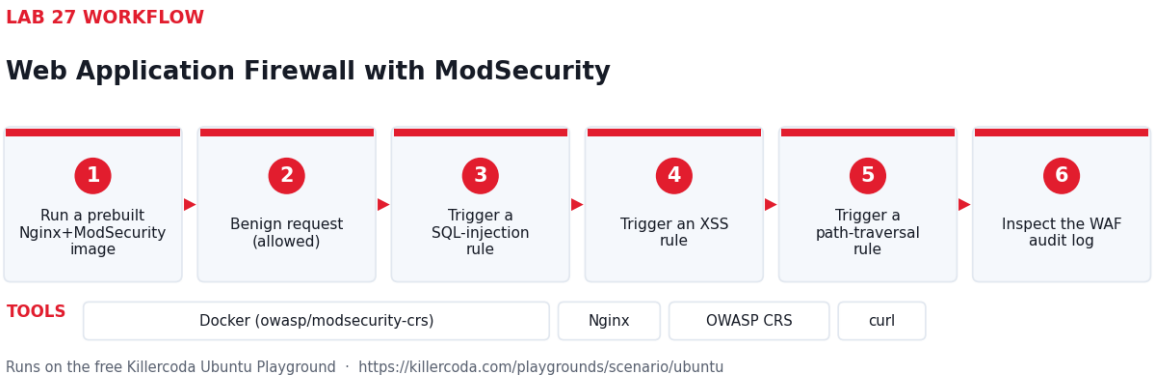

Figure 27 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Docker (owasp/modsecurity-crs), Nginx, OWASP CRS, curl

In this lab you will deploy **Nginx + ModSecurity + the OWASP Core Rule Set (CRS)** and watch it block SQL injection, XSS, and path-traversal attempts.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Run a pre-built Nginx + ModSecurity image

```
apt update && apt install -y docker.io curl
systemctl start docker

docker run -d --name waf \
  -p 80:80 \
  -e BACKEND=http://example.com \
  -e PARANOIA=1 \
  owasp/modsecurity-crs:nginx
sleep 5
curl -sI http://localhost/
```

This image bundles Nginx + libmodsecurity + OWASP CRS — the same WAF logic AWS WAF and Cloudflare WAF base their managed rules on.

Step 2 — Run a benign request (allowed)

```
curl -s -o /dev/null -w "%{http_code}\n" "http://localhost/?q=cats"
```

Step 3 — Trigger a SQL injection rule

```
curl -s -o /dev/null -w "%{http_code}\n" "http://localhost/?id=1' OR '1'='1"
```

You should see **403** — CRS rule 942100 (SQLi).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 4 — Trigger an XSS rule

```
curl -s -o /dev/null -w "%{http_code}\n" "http://localhost/?
msg=<script>alert(1)</script>"
```

403 — CRS rule 941100 (XSS).

Step 5 — Trigger a path-traversal / LFI rule

```
curl -s -o /dev/null -w "%{http_code}\n"
"http://localhost/?file=../../../../etc/passwd"
```

403 — CRS rule 930100.

Step 6 — Inspect the WAF audit log

```
docker exec waf tail -40 /var/log/modsec/modsec_audit.log
```

You will see the matched rule IDs, the request, and the score.

Step 7 — Tune (false-positive triage)

ModSecurity uses an **anomaly score**. The default block threshold is 5. To accept a known-good pattern:

```
docker exec waf sh -c "echo 'SecRuleRemoveById 941100' >>
/etc/modsecurity.d/exclusions.conf"
docker restart waf
```

In production you tune via test → staging → prod, never prod-first.

Step 8 — Network ACL vs WAF vs Security Group

Layer	Inspects	Tool used
Network ACL (Lab 3)	IP/port	iptables
Security Group (Lab 3)	IP/port stateful	iptables -m conntrack
WAF (this lab)	HTTP semantics	ModSecurity / CRS

A WAF sees what a firewall cannot — request bodies, query strings, HTTP headers.

Step 9 — Cleanup

```
docker rm -f waf
```

Test it

Run these checks to prove the lab worked before you move on:

```
docker ps --filter name=waf --format '{{.Names}}\t{{.Status}}'
curl -s -o /dev/null -w "benign=%{http_code}\n" "http://localhost/?q=cats"
curl -s -o /dev/null -w "sqli=%{http_code}\n" "http://localhost/?id=1' OR
'1'='1"
curl -s -o /dev/null -w "xss=%{http_code}\n" "http://localhost/?
msg=<script>alert(1)</script>"
curl -s -o /dev/null -w "lfi=%{http_code}\n" "http://localhost/?
file=../../../../etc/passwd"
docker exec waf tail -20 /var/log/modsec/modsec_audit.log
```

Expected: Run this before Step 9. The `waf` container is **Up**; the benign request returns `benign=200` while all three attacks return **403** (`sqli=403`, `xss=403`, `lfi=403`); and the

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

audit log tail shows the matching OWASP CRS rule IDs — 942100 for SQLi, 941100 for XSS and 930100 for path traversal — together with the anomaly score that crossed the threshold of 5.

What you learned

- WAFs operate on HTTP semantics, not just IP/port.
- OWASP CRS catches the OWASP Top 10 out of the box.
- Anomaly scoring + tuning is the production workflow.

Free tools used

- ModSecurity — https://github.com/owasp-modsecurity/ModSecurity
- OWASP Core Rule Set — https://coreruleset.org
- Cloudflare WAF (free tier) — https://www.cloudflare.com/application-services/products/waf

Lab 28 — Detecting Suspicious Activity with Falco

Goal: Run Falco and trigger runtime detections that map to CV0-004 attack categories, then route alerts to Prometheus / SIEM.

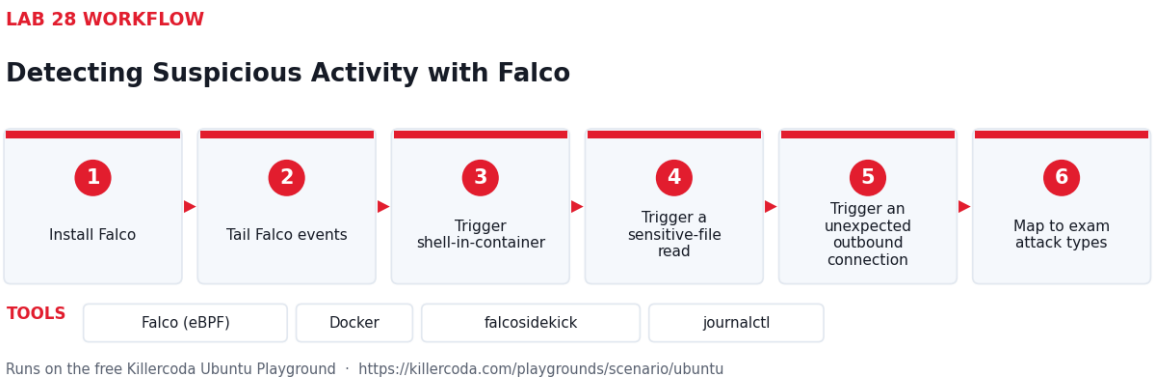

Figure 28 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Falco (eBPF), Docker, falcosidekick, journalctl

In this lab you will run **Falco** — a CNCF runtime security tool — and trigger detections for shell-in-container, sensitive-file reads, unexpected outbound connections, and cryptojacking-style behaviour.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

> ***Web tool:*** *Explore how these attacks look from the defender's side with the free browser* ***Cybersecurity Simulator*** *— https://alfredang.github.io/cybersecuritysimulator/.*

Step 1 — Install Falco (modern eBPF mode)

```
apt update && apt install -y curl gnupg lsb-release docker.io
systemctl start docker

curl -fsSL https://falco.org/repo/falcosecurity-packages.asc | gpg --dearmor
-o /usr/share/keyrings/falco-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/falco-archive-keyring.gpg]
https://download.falco.org/packages/deb stable main" >
/etc/apt/sources.list.d/falcosecurity.list
apt update
FALCO_DRIVER_CHOICE=modern_ebpf apt install -y falco
falco --version
```

If kernel headers are missing on Killercoda, run Falco in **userspace** mode in a container:

```
docker run -d --name falco --privileged \
  -v /var/run/docker.sock:/host/var/run/docker.sock \
  -v /proc:/host/proc:ro \
  falcosecurity/falco-no-driver:latest /usr/bin/falco -o engine.kind=ebpf
```

Step 2 — Tail Falco events

```
journalctl -fu falco &
JLP=$!
```

(or `docker logs -f falco &`)

Step 3 — Trigger: shell spawned in a container

```
docker run -d --name target nginx:alpine
docker exec target sh -c 'echo "shell inside container"'
```

Falco fires:

```
Notice A shell was spawned in a container ... container_id=...
```

Step 4 — Trigger: read of a sensitive file

```
docker exec target cat /etc/shadow
```

Falco rule **Read sensitive file untrusted** fires.

Step 5 — Trigger: unexpected outbound connection

```
docker exec target sh -c 'wget -q http://example.com -O /dev/null'
```

Falco rule **Unexpected outbound connection** fires (in default rules).

Step 6 — Map to attack types from the exam

Detection	Maps to exam category
Shell in container	Vulnerability exploitation
/etc/shadow read	Privilege escalation
Outbound connect	Cryptojacking / zombie instance

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Sudden CPU spike (Lab 16)	Cryptojacking
Unexpected metadata API call	Metadata abuse (cloud-specific)

Step 7 — Baseline deviation alerting

Fold Falco events into Prometheus (Lab 16) via `falcosidekick`:

```
docker run -d --name fsk -p 2801:2801 \
  -e FALCO_LISTEN=0.0.0.0:2801 \
  falcosecurity/falcosidekick
```

Then in `falco.yaml` set `http_output: enabled: true, url: http://falcosidekick:2801`.

Step 8 — Cleanup

```
kill $JLP 2>/dev/null
docker rm -f target falco fsk 2>/dev/null
```

Test it

Run these checks to prove the lab worked before you move on:

```
falco --version || docker ps --filter name=falco --format '{{.Names}}
\t{{.Status}}'
journalctl -u falco -n 30 --no-pager 2>/dev/null || docker logs --tail 30
falco
docker ps --filter name=target --format '{{.Names}}\t{{.Status}}'
curl -sI http://localhost:2801/ping | head -1
```

Expected: Run this before Step 8. Falco reports its version (or the container is **Up**); the event log contains one line per trigger — `A shell was spawned in a container`, `Read sensitive file untrusted` for the `/etc/shadow` read, and `Unexpected outbound connection` for the wget — each tagged with the `target` container's ID; and falcosidekick answers on port 2801.

What you learned

- Runtime detection uses kernel signals (eBPF) to spot bad behaviour.
- Common detections map to CV0-004 attack categories.
- Falco events flow into Prometheus / SIEM.

Free tools used

- Falco — https://falco.org
- Falcosidekick — https://github.com/falcosecurity/falcosidekick
- auditd (built-in) — https://github.com/linux-audit/audit-userspace
- Wazuh (free SIEM) — https://wazuh.com

Domain 5 — DevOps Fundamentals (10%)

Source control and branching, CI/CD pipelines, web-service APIs (REST / GraphQL) and container orchestration with Kubernetes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What this domain covers in the labs:

- **Lab 29 — Git Source Control & Branching:** Exercise every CV0-004 source-control verb — commit, push, branch, merge, PR review, conflict resolution — against a self-hosted Gitea server.
- **Lab 30 — CI/CD Pipeline with GitHub Actions (act):** Build and run a local test → build → scan → deploy CI/CD pipeline with act, mapping each stage to CV0-004 CI/CD concepts.
- **Lab 31 — REST & GraphQL APIs:** Call a public REST API, host a local GraphQL server, stream over WebSockets and preview pub/sub event-driven messaging.
- **Lab 32 — Container Orchestration with Kubernetes (k3s):** Install k3s and deploy, expose, scale, roll out/rollback, add persistence, self-healing and RBAC on a real Kubernetes cluster.

Lab 29 — Git Source Control & Branching

Goal: Exercise every CV0-004 source-control verb — commit, push, branch, merge, PR review, conflict resolution — against a self-hosted Gitea server.

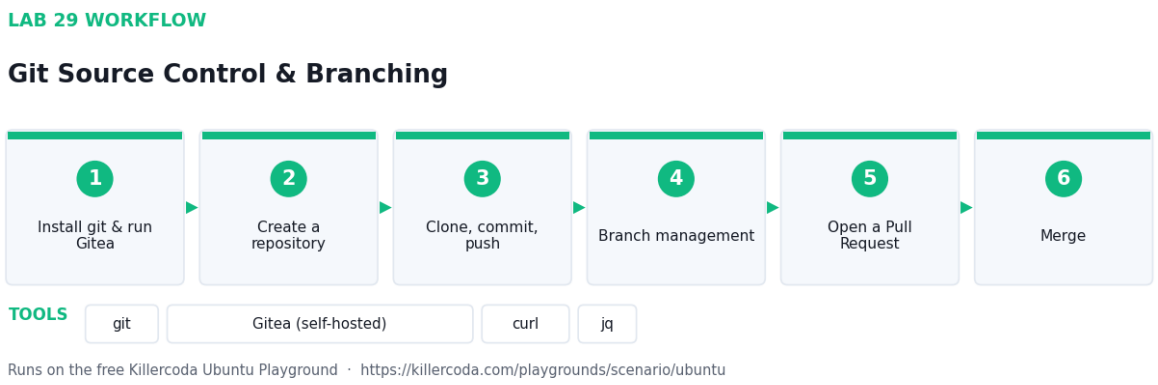

Figure 29 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: git, Gitea (self-hosted), curl, jq

In this lab you will exercise every CV0-004 source-control verb: **commit, push, branch, merge, pull request review** — using a local Gitea server as your "GitHub clone".

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install git and run Gitea (self-hosted)

```
apt update && apt install -y git docker.io curl jq
systemctl start docker
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker run -d --name gitea -p 3000:3000 -p 2222:22 gitea/gitea:latest
sleep 25
curl -sI http://localhost:3000 | head -1
```

UI: `http://<killercoda-host>:3000` — set up admin user (e.g. `admin`/`cloudplus`/`a@x.com`) on first visit.

Step 2 — Create a repository (UI or API)

```
curl -s -u admin:cloudplus -X POST http://localhost:3000/api/v1/user/repos \
  -H 'Content-Type: application/json' \
  -d '{"name":"infra","auto_init":true,"default_branch":"main"}' |
jq .full_name
```

Step 3 — Clone, commit, push

Ready-made file: `main.tf` — you can download it instead of typing this block.

```
mkdir -p /tmp/work && cd /tmp/work
git clone http://admin:cloudplus@localhost:3000/admin/infra.git
cd infra
git config user.email lab@x
git config user.name lab

cat > main.tf <<'EOF'
resource "aws_s3_bucket" "data" { bucket = "main-bucket" }
EOF
git add main.tf
git commit -m "feat: initial bucket"
git push origin main
```

Step 4 — Branch management

Ready-made file: `encryption.tf.snippet` — you can download it instead of typing this block.

```
git checkout -b feature/encryption
cat >> main.tf <<'EOF'
resource "aws_s3_bucket_server_side_encryption_configuration" "data" {
  bucket = aws_s3_bucket.data.id
  rule { apply_server_side_encryption_by_default { sse_algorithm =
"AES256" } }
}
EOF
git commit -am "feat: add SSE"
git push origin feature/encryption
```

Step 5 — Open a Pull Request

```
curl -s -u admin:cloudplus -X POST
http://localhost:3000/api/v1/repos/admin/infra/pulls \
  -H 'Content-Type: application/json' \
  -d '{"title":"Add
SSE","head":"feature/encryption","base":"main","body":"Encrypts the bucket"}'
```

Open the PR in the UI and review the diff — that is the **code review** sub-objective.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 6 — Merge

```
PR=1
curl -s -u admin:cloudplus -X POST
http://localhost:3000/api/v1/repos/admin/infra/pulls/$PR/merge \
  -H 'Content-Type: application/json' \
  -d '{"Do":"merge"}'

git checkout main
git pull
cat main.tf | tail -5
```

Step 7 — Resolve a merge conflict

```
git checkout -b feature/region
sed -i 's/main-bucket/main-bucket-us/' main.tf
git commit -am "feat: rename to us"
git push origin feature/region

git checkout main
sed -i 's/main-bucket/main-bucket-eu/' main.tf
git commit -am "feat: rename to eu"
git push origin main

git merge feature/region || true
grep -n '<<<' main.tf
# resolve
sed -i 's/<<<<<<<.*//; s/=======.*//; s/>>>>>>>.*//' main.tf
git commit -am "merge: keep eu"
```

Step 8 — Branch protection (concept)

In production, protect `main`:

- Require PR + 1 approval.
- Require CI green.
- Block force-push.

These map to the exam's **branch management** sub-objective.

Step 9 — Cleanup

```
docker rm -f gitea
rm -rf /tmp/work
```

Test it

Run these checks to prove the lab worked before you move on:

```
curl -sI http://localhost:3000 | head -1
curl -s -u admin:cloudplus
http://localhost:3000/api/v1/repos/admin/infra/branches | jq '.[].name'
curl -s -u admin:cloudplus
http://localhost:3000/api/v1/repos/admin/infra/pulls?state=all | jq '.[] |
{title, state}'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cd /tmp/work/infra && git log --oneline --graph --all | head -15
grep -c '<<<<<<<' main.tf
```

Expected: Run this before Step 9. Gitea returns `HTTP/1.1 200 OK`; the branch list contains `main`, `feature/encryption` and `feature/region`; the PR query shows `{"title":"Add SSE","state":"closed"}` — closed because it was merged; the commit graph shows the feature branches joining `main` at a merge commit; and `grep -c` returns `0`, proving the conflict markers were resolved before the final commit.

What you learned

- Commit → push → branch → PR → merge.
- Conflict resolution is a normal Git workflow.
- Branch protection enforces the review policy.

Free tools used

- git (built-in) — https://git-scm.com
- Gitea — https://about.gitea.com
- GitHub (free public repos) — https://github.com
- GitLab Community — https://about.gitlab.com
- Pro Git book (free) — https://git-scm.com/book

Files in this lab

File	Purpose
`main.tf`	Step 3 initial Terraform file committed to the Gitea repo.
`encryption.tf.snippet`	Step 4 block appended to `main.tf` on the `feature/encryption` branch.

Lab 30 — CI/CD Pipeline with GitHub Actions (act)

Goal: Build and run a local test → build → scan → deploy CI/CD pipeline with act, mapping each stage to CV0-004 CI/CD concepts.

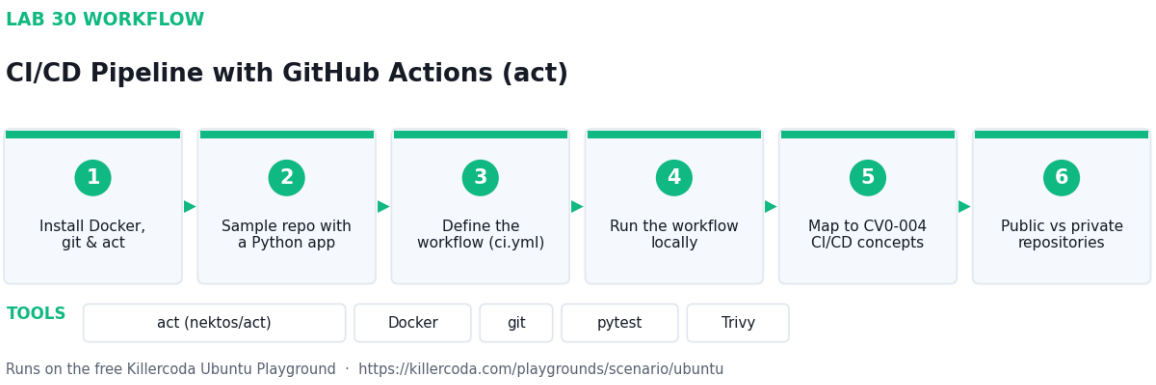

Figure 30 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Tools used: act (nektos/act), Docker, git, pytest, Trivy

In this lab you will build a CI/CD pipeline locally with **act** — a free tool that runs GitHub Actions workflows in Docker. The workflow tests, builds, scans, and "deploys" an artifact.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install Docker, git, and act

```
apt update && apt install -y docker.io git curl
systemctl start docker

curl -s https://raw.githubusercontent.com/nektos/act/master/install.sh | bash
-s -- -b /usr/local/bin
act --version
```

Step 2 — Sample repository with a Python app

Ready-made files: `app.py`, `test_app.py` and `Dockerfile` — you can download them instead of typing this block.

```
mkdir -p /tmp/cicd && cd /tmp/cicd
git init -q -b main

cat > app.py <<'EOF'
def add(a,b): return a+b
EOF

cat > test_app.py <<'EOF'
from app import add
def test_add(): assert add(2,3) == 5
EOF

cat > Dockerfile <<'EOF'
FROM python:3.12-slim
COPY app.py /app/app.py
WORKDIR /app
CMD ["python", "-c", "from app import add; print(add(2,3))"]
EOF
```

Step 3 — Define the workflow (.github/workflows/ci.yml)

Ready-made file: `ci.yml` — you can download it instead of typing this block.

```
mkdir -p .github/workflows
cat > .github/workflows/ci.yml <<'EOF'
name: ci
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
      - uses: actions/checkout@v4
      - name: install
        run: pip install pytest
      - name: unit tests
        run: pytest -q
  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: build image
        run: docker build -t myapp:${{ github.sha }} .
  scan:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: trivy fs
        run: |
          curl -sSL https://aquasecurity.github.io/trivy/install.sh | sh -s --
-b /usr/local/bin
          trivy fs --severity CRITICAL --exit-code 0 .
  deploy:
    needs: scan
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying $GITHUB_SHA to staging"
EOF

git add . && git -c user.email=lab@x -c user.name=lab commit -q -m "ci"
```

Step 4 — Run the workflow locally

```
act -j test
```

You will see Docker pull a runner image, install pytest, run the tests — green.

```
act -j build
act -j scan
act -j deploy
```

Or run the whole pipeline:

```
act
```

Step 5 — Map to CV0-004 CI/CD concepts

Stage in your YAML	Exam term
`test`	Testing
`build`	Code integration + Build
`scan`	Security
`deploy`	Code deployment
`myapp:${{github.sha}}`	Artifact (container image)
`Dockerfile`	Packaging

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 6 — Public vs private repositories

Push to GitHub:

```
# git remote add origin https://github.com/<you>/<repo>.git
# git push -u origin main
```

The same workflow runs on GitHub's hosted runners — public free for OSS, private free up to 2000 minutes/month.

Step 7 — Other free CI options

Tool	Type	Notes
GitHub Actions	SaaS	2000 free min/month private
GitLab CI	SaaS / self-hosted	400 free CI min
Jenkins	Self-hosted	classic, plugin-rich
Drone CI	Self-hosted	Docker-native
Tekton	Self-hosted on K8s	cloud-native

Step 8 — Cleanup

```
rm -rf /tmp/cicd
```

Test it

Run these checks to prove the lab worked before you move on:

```
act --version
cd /tmp/cicd && ls .github/workflows/ci.yml
act -l
act -j test
git -C /tmp/cicd log --oneline
```

Expected: Run this before Step 8. `act` prints its version; the workflow file exists; `act -l` lists all four jobs (`test`, `build`, `scan`, `deploy`) with their `needs` dependencies; re-running `act -j test` ends with a green `**Job succeeded**` line after `pytest` reports `1 passed`; and the repo has the `ci` commit.

What you learned

- A pipeline is YAML + jobs + dependencies.
- act lets you iterate without pushing.
- Test → build → scan → deploy is the canonical flow.

Free tools used

- act — https://github.com/nektos/act
- GitHub Actions — https://github.com/features/actions
- GitLab CI — https://docs.gitlab.com/ee/ci
- Jenkins — https://www.jenkins.io
- Drone — https://www.drone.io

Files in this lab

File	Purpose
`app.py`	Step 2 application code under test.
`test_app.py`	Step 2 pytest unit test run by the `test` job.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

`Dockerfile`	Step 2 packaging definition built by the `build` job.
`ci.yml`	Step 3 GitHub Actions workflow (test → build → scan → deploy); copy to `.github/workflows/ci.yml`.

Lab 31 — REST & GraphQL APIs

Goal: Call a public REST API, host a local GraphQL server, stream over WebSockets and preview pub/sub event-driven messaging.

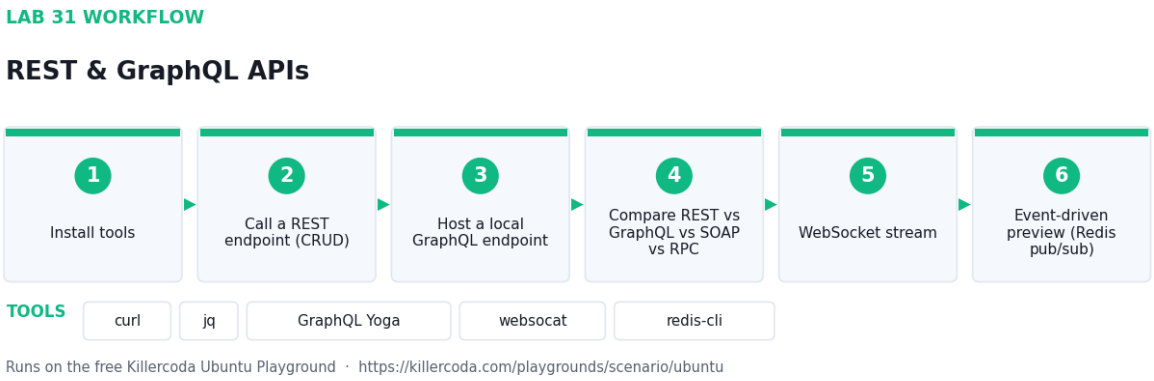

Figure 31 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: curl, jq, GraphQL Yoga, websocat, redis-cli

In this lab you will call a public **REST** API, host a local **GraphQL** server, and try a **WebSocket** stream — covering the exam's web-services and integration sub-objectives.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install tools

```
apt update && apt install -y curl jq docker.io websocat
systemctl start docker
```

Step 2 — Call a REST endpoint (CRUD)

The free **JSONPlaceholder** API supports GET/POST/PUT/DELETE.

```
curl -s https://jsonplaceholder.typicode.com/posts/1 | jq

# Create
curl -s -X POST https://jsonplaceholder.typicode.com/posts \
  -H 'Content-Type: application/json' \
  -d '{"title":"hello","body":"cloud","userId":1}' | jq
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
# Update
curl -s -X PUT https://jsonplaceholder.typicode.com/posts/1 \
  -H 'Content-Type: application/json' \
  -d '{"id":1,"title":"changed","body":"x","userId":1}' | jq

# Delete
curl -sI -X DELETE https://jsonplaceholder.typicode.com/posts/1 | head -1
```

REST = stateless verbs over HTTP, one resource per URL.

Step 3 — Host a local GraphQL endpoint

Ready-made file: `server.mjs` — you can download it instead of typing this block.

```
docker run -d --name gql -p 4000:4000 \
  graphql/swapi-graphql 2>/dev/null || \
docker run -d --name gql -p 4000:4000 \
  -e PORT=4000 node:20-alpine sh -c '
    npm i -g graphql-yoga@4 graphql >/dev/null
    cat > /tmp/server.mjs <<"JS"
import { createYoga, createSchema } from "graphql-yoga"
import { createServer } from "node:http"
const yoga = createYoga({ schema: createSchema({
  typeDefs: `type Query { hello(name: String): String }`,
  resolvers: { Query: { hello: (_,a)=>"Hello "+(a.name||"cloud") } }
})})
createServer(yoga).listen(4000)
JS
    node /tmp/server.mjs'
sleep 8
```

Send a GraphQL query:

```
curl -s http://localhost:4000/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"{ hello(name:\"Cloud+\") }"}' | jq
```

GraphQL = single endpoint, client picks fields, fewer over-fetches.

Step 4 — REST vs GraphQL vs SOAP vs RPC

Style	Verb	Payload	Schema
REST	GET/POST/...	JSON	OpenAPI
SOAP	POST	XML	WSDL
GraphQL	POST	JSON	GraphQL SDL
gRPC (RPC)	HTTP/2	Protobuf	.proto

Step 5 — WebSocket stream (real-time)

```
docker run -d --name echo -p 8765:8080 \
  jmalloc/echo-server
sleep 2

echo "hello-stream" | websocat -n1 ws://localhost:8765/.ws
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

WebSockets keep the TCP connection open — used by chat, dashboards, K8s `kubectl exec`.

Step 6 — Event-driven preview

```
docker run -d --name redis -p 6379:6379 redis:7-alpine
sleep 2
( for i in 1 2 3; do
    docker exec redis redis-cli PUBLISH events "msg-$i"
    sleep 1
  done ) &
docker exec redis sh -c 'timeout 5 redis-cli SUBSCRIBE events'
```

Pub/sub = event-driven architecture (Kafka, SNS/SQS, NATS use the same pattern).

Step 7 — Cleanup

```
docker rm -f gql echo redis 2>/dev/null
```

Test it

Run these checks to prove the lab worked before you move on:

```
curl -s https://jsonplaceholder.typicode.com/posts/1 | jq '.id, .title'
curl -s http://localhost:4000/graphql -H 'Content-Type: application/json' -d
'{"query":"{ hello(name:\"Cloud+\") }"}' | jq
echo "hello-stream" | websocat -n1 ws://localhost:8765/.ws
docker ps --filter name=gql --filter name=echo --filter name=redis --format
'{{.Names}}\t{{.Status}}'
```

Expected: Run this before Step 7. The REST call returns post `1` with its title (stateless GET over HTTP); the GraphQL query returns exactly `{"data":{"hello":"Hello Cloud+"}}` — only the field you asked for, nothing more; `websocat` echoes `hello-stream` back over the open WebSocket; and all three containers (`gql`, `echo`, `redis`) are **Up**.

What you learned

- REST CRUD with curl.
- GraphQL queries return only requested fields.
- WebSockets for streams; pub/sub for events.

Free tools used

- jsonplaceholder.typicode.com — public REST sandbox
- GraphQL Yoga — https://the-guild.dev/graphql/yoga-server
- websocat — https://github.com/vi/websocat
- Postman free tier — https://www.postman.com
- Hoppscotch (open-source) — https://hoppscotch.io

Files in this lab

File	Purpose
`server.mjs`	Step 3 GraphQL Yoga server exposing the `hello(name)` query.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 32 — Container Orchestration with Kubernetes (k3s)

Goal: Install k3s and deploy, expose, scale, roll out/rollback, add persistence, self-healing and RBAC on a real Kubernetes cluster.

Figure 32 — the workflow you build in this lab.

Runs on: Killercoda Kubernetes Playground — https://killercoda.com/playgrounds/scenario/kubernetes

Tools used: k3s, kubectl, YAML manifests, curl

In this lab you will install **k3s** (a minimal, single-binary Kubernetes), deploy an app, expose it, scale it, and watch a rolling update — the canonical workload-orchestration platform on every cloud.

Lab platform

Run this lab on the **Killercoda Kubernetes Playground** — it gives you a real cluster with `kubectl` already configured:

https://killercoda.com/playgrounds/scenario/kubernetes

> *The Ubuntu playground has no cluster. If you prefer to work locally, enable Kubernetes in **Docker Desktop** (https://www.docker.com/products/docker-desktop/) instead.*

Step 1 — Install k3s

```
apt update && apt install -y curl
curl -sfL https://get.k3s.io | sh -
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
kubectl get nodes
```

Step 2 — Deploy an application

Ready-made file: `app.yaml` — you can download it instead of typing this block.

```
cat > app.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
metadata: { name: web }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
      - name: web
        image: nginx:1.26-alpine
        ports: [{ containerPort: 80 }]
---
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  type: NodePort
  selector: { app: web }
  ports: [{ port: 80, nodePort: 30080 }]
EOF

kubectl apply -f app.yaml
kubectl rollout status deploy/web
kubectl get pods,svc
curl -s http://localhost:30080 | head -3
```

Step 3 — Scale (horizontal)

```
kubectl scale deploy/web --replicas=5
kubectl get pods -l app=web
```

This is the **horizontal scaling** primitive from Lab 19, but managed.

Step 4 — Rolling update (Lab 12 idea, K8s-native)

```
kubectl set image deploy/web web=nginx:1.27-alpine
kubectl rollout status deploy/web
kubectl rollout history deploy/web
```

Rollback if anything breaks:

```
kubectl rollout undo deploy/web
```

Step 5 — Persistent volume

Ready-made file: `pvc.yaml` — you can download it instead of typing this block.

```
cat > pvc.yaml <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 100Mi } }
EOF
kubectl apply -f pvc.yaml
kubectl get pvc
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

k3s ships with `local-path` provisioner — equivalent to AWS EBS / Azure Disk.

Step 6 — Liveness probe (self-healing)

```
kubectl patch deploy/web --type=json -p '[
  {"op":"add","path":"/spec/template/spec/containers/0/livenessProbe",
   "value":{"httpGet":
{"path":"/","port":80},"initialDelaySeconds":3,"periodSeconds":5}}
]'
kubectl get pods -l app=web
```

If a pod fails its probe, the kubelet restarts it. **Self-healing**.

Step 7 — RBAC (Lab 24 idea, K8s-native)

```
kubectl create role pod-reader --verb=get,list --resource=pods
kubectl create rolebinding alice-pods --role=pod-reader --user=alice
kubectl auth can-i list pods --as alice
kubectl auth can-i delete pods --as alice
```

Ready-made manifests

Every object above is also available as a proper, apply-ready YAML file in the `manifests/` folder — clone or download them instead of typing the inline YAML and `kubectl create` commands:

File	Lab step	Apply with
`manifests/deployment.yaml`	Step 2 — the `web` Deployment (3 × `nginx:1.26-alpine`)	`kubectl apply -f manifests/deployment.yaml`
`manifests/service.yaml`	Step 2 — the NodePort Service on 30080	`kubectl apply -f manifests/service.yaml`
`manifests/pvc.yaml`	Step 5 — the `data` PersistentVolumeClaim	`kubectl apply -f manifests/pvc.yaml`
`manifests/liveness.yaml`	Step 6 — the Deployment with the liveness probe set	`kubectl apply -f manifests/liveness.yaml`
`manifests/rbac.yaml`	Step 7 — the `pod-reader` Role + `alice-pods` RoleBinding	`kubectl apply -f manifests/rbac.yaml`

Or apply the whole set at once:

```
kubectl apply -f manifests/
```

See `manifests/README.md` for details.

Step 8 — Map K8s ↔ exam objectives

K8s primitive	CV0-004 concept
Deployment	Workload orchestration
Service	Application/network load balancer
ConfigMap / Secret	CaC + Secrets management
HPA	Horizontal scaling
PVC	Persistent volume
RBAC	Authorization model
NetworkPolicy	Security group

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 9 — Cleanup

```
kubectl delete -f app.yaml
kubectl delete pvc data
/usr/local/bin/k3s-uninstall.sh
```

Test it

Run these checks to prove the lab worked before you move on:

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
kubectl get nodes
kubectl get deploy,pods,svc -l app=web
kubectl rollout history deploy/web
kubectl get pvc data
kubectl auth can-i list pods --as alice
curl -s http://localhost:30080 | head -3
```

Expected: Run this before Step 9. The node reports status **Ready**; the `web` Deployment shows `5/5` ready pods (after the Step 3 scale) all in `Running`; `rollout history` lists at least two revisions from the `nginx:1.26-alpine` → `1.27-alpine` update; the `data` PVC is **Bound** to a `local-path` volume; `kubectl auth can-i list pods --as alice` answers **yes** (while `delete pods` answers **no**); and the NodePort on 30080 returns the nginx welcome HTML.

What you learned

- k3s gives you a real cluster in one shell command.
- Deployments, services, scaling, rolling updates, RBAC.
- K8s implements every cloud-orchestration primitive on the exam.

Free tools used

- k3s — https://k3s.io
- kubectl — bundled
- Minikube (alternative) — https://minikube.sigs.k8s.io
- k9s (terminal UI) — https://k9scli.io
- Lens Desktop (free) — https://k8slens.dev
- Killercoda K8s playground — https://killercoda.com/playgrounds/scenario/kubernetes

Files in this lab

File	Purpose
`app.yaml`	Step 2 Deployment + NodePort Service applied to the cluster.
`pvc.yaml`	Step 5 PersistentVolumeClaim backed by the local-path provisioner.
`manifests/deployment.yaml`	Step 2 — apply-ready `web` Deployment (3 × nginx:1.26-alpine).
`manifests/service.yaml`	Step 2 — apply-ready NodePort Service on port 30080.
`manifests/pvc.yaml`	Step 5 — apply-ready `data` PersistentVolumeClaim (100Mi).
`manifests/liveness.yaml`	Step 6 — the `web` Deployment with the liveness probe set (self-healing).
`manifests/rbac.yaml`	Step 7 — the `pod-reader` Role and `alice-pods`

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	RoleBinding.
`manifests/README.md`	Index of the manifests with the `kubectl apply -f` command for each.

Domain 6 — Troubleshooting (12%)

Methodically diagnosing and fixing deployment, network, security and container / resource issues using a repeatable triage workflow.

What this domain covers in the labs:

- **Lab 33 — Troubleshoot Deployment Issues:** Reproduce and fix each CV0-004 6.1 deployment failure mode locally — resources, permissions, oversubscription, sizing, deprecation, quotas, regions.
- **Lab 34 — Troubleshoot Cloud Network Issues:** Reproduce and resolve CV0-004 6.2 network failures using an L2→L7 diagnostic ladder (DHCP, DNS, NTP, NAT, HTTP codes, routing, IP overlap).
- **Lab 35 — Troubleshoot TLS, Cipher & Auth Issues:** Reproduce and fix CV0-004 6.3 security failures from the CLI — deprecated ciphers, privesc, unauthorized access, leaked credentials, vulnerabilities.
- **Lab 36 — Troubleshoot Container & Resource Limits:** Combine the Lab 33–35 methods to triage real container failures with a repeatable state → logs → stats → mounts → network workflow.

Lab 33 — Troubleshoot Deployment Issues

Goal: Reproduce and fix each CV0-004 6.1 deployment failure mode locally — resources, permissions, oversubscription, sizing, deprecation, quotas, regions.

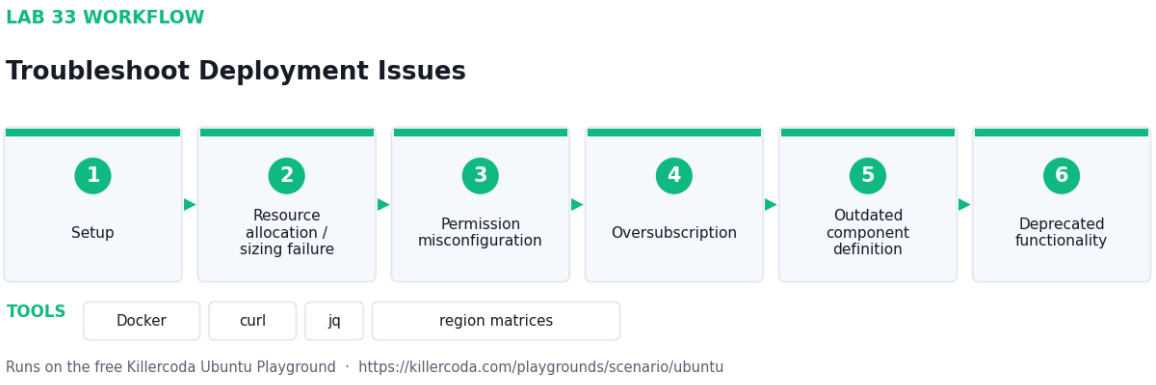

Figure 33 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: Docker, curl, jq, region matrices

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

In this lab you will reproduce and fix the deployment problems listed in CV0-004 6.1: **resource allocation, permissions, oversubscription, sizing, outdated definitions, deprecation, quotas, regional availability**.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Setup

```
apt update && apt install -y docker.io curl jq
systemctl start docker
```

Step 2 — Resource allocation / sizing failure

```
docker run --rm --memory=10m alpine sh -c "yes | head -c 50m" 2>&1 | head -3
```

The container is OOM-killed because we under-sized memory.

```
docker stats --no-stream
```

Fix: raise the limit:

```
docker run --rm --memory=128m alpine sh -c "yes | head -c 50m >/dev/null &&
echo OK"
```

Step 3 — Permission misconfiguration

```
docker run -d --name pg --read-only -e POSTGRES_PASSWORD=cloud postgres:16
2>&1 | tail -3
docker logs pg 2>&1 | tail -5
docker rm -f pg
```

Postgres needs a writable volume; `--read-only` blocks startup.

Fix: mount a writable tmpfs/volume:

```
docker run -d --name pg --read-only --tmpfs /var/run/postgresql --tmpfs /tmp \
  -v pgdata:/var/lib/postgresql/data -e POSTGRES_PASSWORD=cloud postgres:16
sleep 5 && docker logs pg | tail -3
docker rm -f pg && docker volume rm pgdata
```

Step 4 — Oversubscription

```
docker run --rm --cpus=0.1 alpine sh -c "for i in 1 2 3 4 5; do dd
if=/dev/zero of=/dev/null bs=1M count=200 & done; wait" 2>&1 | tail -3
```

CPU oversubscribed — work piles up in run queue. **Fix:** size CPU to peak load, not average.

Step 5 — Outdated component definition (image)

```
docker pull alpine:3.10 2>&1 | tail -1
docker run --rm alpine:3.10 cat /etc/alpine-release
```

3.10 is end-of-life — outdated definition. **Fix:** pin to a supported tag (`alpine:3.20`).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 6 — Deprecated functionality

```
docker run --rm alpine:3.20 sh -c 'echo "iptables-legacy still works:"; apk
add --quiet iptables; iptables -L 2>&1 | head -3'
```

Some flags in `docker run` are deprecated (`--link`). Always check the changelog when an upgrade fails.

Step 7 — Outage / partial outage simulation

```
docker run -d --name web -p 8080:80 nginx:alpine
docker pause web
curl -m 3 -sI http://localhost:8080 | head -1 || echo "PARTIAL OUTAGE:
container paused"
docker unpause web
curl -sI http://localhost:8080 | head -1
docker rm -f web
```

Step 8 — API throttling / service quota

Cloud APIs return HTTP **429 Too Many Requests** when throttled. Reproduce with a rate-limited responder:

```
docker run -d --name limit -p 8081:80 \
  -e DELAY=0 nginx:alpine

# Hammer it
for i in $(seq 1 50); do curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:8081/; done | sort | uniq -c
docker rm -f limit
```

Fix in real cloud: request a quota increase, add exponential backoff, batch calls.

Step 9 — Regional service availability

Not every cloud service exists in every region. The fix is preventive — read the region matrix:

- AWS — https://aws.amazon.com/about-aws/global-infrastructure/regional-product-services/
- Azure — https://azure.microsoft.com/en-us/explore/global-infrastructure/products-by-region/
- GCP — https://cloud.google.com/about/locations

Step 10 — Decision tree

1. Does the deployment **start**? → permissions, image, definition.
2. Does it **stay up**? → resources, OOM, CPU.
3. Does it **respond**? → quotas, throttling, availability.
4. Did it **work before**? → drift, deprecation, version skew.

Test it

Run these checks to prove the lab worked before you move on:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
docker run --rm --memory=128m alpine sh -c "yes | head -c 50m >/dev/null &&
echo OK"
docker run --rm alpine:3.10 cat /etc/alpine-release
docker ps -a --filter name=web --format '{{.Names}}\t{{.Status}}'
for i in $(seq 1 20); do curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:8081/; done | sort | uniq -c
docker stats --no-stream
```

Expected: Run each check at the point in the lab where its container still exists. The correctly-sized 128m container prints `OK` where the 10m one was OOM-killed; `alpine:3.10` prints `3.10.9` — an end-of-life release; the paused `web` container shows status `Up ... (Paused)` and `curl` times out until you `docker unpause` it and get `HTTP/1.1 200 OK`; and the 20 requests to the un-throttled responder all tally as `200` (a real cloud API would return `429` once the quota is hit).

What you learned

- Reproduce each CV0-004 6.1 failure mode locally.
- Read logs/metrics first, fix the parameter that matches the symptom.
- Deprecation and EoL are deployment risks — track them.

Free tools used

- Docker — https://www.docker.com
- AWS / Azure / GCP region matrices (linked above)

Lab 34 — Troubleshoot Cloud Network Issues

Goal: Reproduce and resolve CV0-004 6.2 network failures using an L2→L7 diagnostic ladder (DHCP, DNS, NTP, NAT, HTTP codes, routing, IP overlap).

Figure 34 — the workflow you build in this lab.

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: dig/nslookup, ip, iptables, mtr, chrony, tcpdump

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

In this lab you will reproduce and resolve the network failures from CV0-004 6.2: **DHCP, DNS, NTP, NAT, HTTP status codes, latency, routing, switching, IP overlap, scope exhaustion**.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

Step 1 — Install diagnostics

```
apt update && apt install -y dnsutils iproute2 iputils-ping curl traceroute
mtr-tiny tcpdump iptables docker.io chrony jq
systemctl start docker
```

Step 2 — DNS failure → fix

```
echo "nameserver 192.0.2.1" > /tmp/bad-resolv.conf
docker run --rm --dns 192.0.2.1 alpine nslookup example.com 2>&1 | tail -4
echo "Symptom: SERVFAIL / no servers reachable"

# Fix: use a valid resolver
docker run --rm --dns 1.1.1.1 alpine nslookup example.com 2>&1 | tail -4
```

Step 3 — NTP / time skew

```
date
hwclock --systohc 2>/dev/null
chronyc tracking 2>/dev/null | head -5 || echo "chrony not running"
chronyc sources 2>/dev/null
```

Clock skew breaks **TLS handshakes**, **MFA tokens**, and **Kerberos**. Always enable an NTP source.

Step 4 — NAT failure

```
ip netns add p
ip link add veth0 type veth peer name veth1
ip link set veth1 netns p
ip addr add 10.99.0.1/24 dev veth0; ip link set veth0 up
ip -n p addr add 10.99.0.2/24 dev veth1; ip -n p link set veth1 up; ip -n p
link set lo up
ip -n p route add default via 10.99.0.1

# Without NAT — fails
ip -n p ping -c 2 -W 2 8.8.8.8 || echo "Symptom: blocked, no NAT"

# Add MASQUERADE
sysctl -w net.ipv4.ip_forward=1
iptables -t nat -A POSTROUTING -s 10.99.0.0/24 -o eth0 -j MASQUERADE
ip -n p ping -c 2 8.8.8.8 || echo "Outbound ping may be blocked by host
policy"

# Cleanup
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
iptables -t nat -D POSTROUTING -s 10.99.0.0/24 -o eth0 -j MASQUERADE
ip netns del p
```

Step 5 — HTTP status code triage

```
docker run -d --name web -p 8080:80 nginx:alpine
curl -s -o /dev/null -w "URL=%{url_effective} HTTP=%{http_code}\n"
http://localhost:8080/missing
curl -s -o /dev/null -w "HTTP=%{http_code}\n" http://localhost:8080/
docker rm -f web
```

Code	Meaning	Likely cause
4xx	Client	Wrong URL, auth, payload
5xx	Server	Backend down, TLS, misconfig
502/504	LB → upstream	Upstream not responding
429	Rate limited	Quota / WAF

Step 6 — Latency / bandwidth (Lab 35 of N+ revisited)

```
ping -c 4 1.1.1.1 | tail -2
mtr -rwc 5 1.1.1.1 || true
```

If RTT > expected, traceroute hop-by-hop to find the slow link.

Step 7 — IP overlap & scope exhaustion

```
ip route show | grep -E '(192\.168|10\.|172\.)'
echo "If two VPCs/peers share 10.0.0.0/24 — NAT or re-IP one side."
echo "Scope exhaustion: DHCP pool is full → enlarge the subnet or shorten the
lease."
```

Step 8 — Routing issues

```
ip route get 8.8.8.8
ip route get 10.255.255.255 || true
```

`ip route get` shows which route the kernel selects. Missing or wrong default → no internet.

Step 9 — VLAN / switching issues (concept)

Two common symptoms:

- **Mismatched access vs trunk** — a tagged frame hits an access port → drop.
- **Wrong native VLAN** — frames leak between segments.

In cloud, the equivalent is wrong **subnet** → **route table** association.

Step 10 — Decision tree

1. Local stack OK? `ip a`, `ss -tlnp`
2. ARP / neighbour OK? `ip neigh`
3. Default route OK? `ip route`
4. DNS OK? `dig`
5. Path OK? `mtr`

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

6. Service OK? `curl -v`

Test it

Run these checks to prove the lab worked before you move on:

```
docker run --rm --dns 1.1.1.1 alpine nslookup example.com | tail -3
chronyc tracking 2>/dev/null | head -5 || timedatectl
ip route get 8.8.8.8
curl -s -o /dev/null -w "root=%{http_code}\n" http://localhost:8080/
curl -s -o /dev/null -w "missing=%{http_code}\n" http://localhost:8080/missing
ping -c 4 1.1.1.1 | tail -2
```

Expected: Run the HTTP checks while the `web` container from Step 5 is still running. The good resolver returns an `Address:` line for `example.com` (the `192.0.2.1` resolver returned SERVFAIL / no servers reachable); chrony or `timedatectl` shows a synchronised clock; `ip route get 8.8.8.8` names the outgoing interface and `via` gateway the kernel selected; the two curls return `root=200` and `missing=404` — the 4xx localises the fault to the client request, not the server; and the ping summary shows `0% packet loss` with an average RTT.

What you learned

- A ladder of checks from L2 → L7.
- HTTP status codes localise the failure layer.
- NTP and DNS break "everything else" when they break.

Free tools used

- dig / nslookup / mtr / traceroute / ip — built-in
- chrony — https://chrony-project.org
- TIS PCAP Analyzer — https://alfredang.github.io/pcapanalyzer/
- HTTP status reference — https://httpstatuses.com

Lab 35 — Troubleshoot TLS, Cipher & Auth Issues

Goal: Reproduce and fix CV0-004 6.3 security failures from the CLI — deprecated ciphers, privesc, unauthorized access, leaked credentials, vulnerabilities.

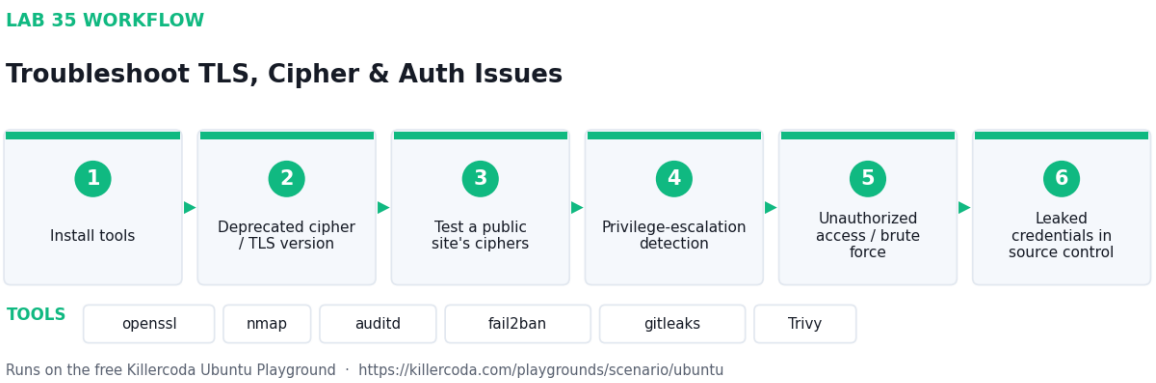

Figure 35 — the workflow you build in this lab.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Runs on: Killercoda Ubuntu Playground — https://killercoda.com/playgrounds/scenario/ubuntu

Tools used: openssl, nmap, auditd, fail2ban, gitleaks, Trivy

In this lab you will reproduce and fix the security failures from CV0-004 6.3: **deprecated cipher suites, privilege escalation, unauthorized access, leaked credentials, software vulnerabilities, unauthorized software**.

Lab platform

Run all commands on the **Killercoda Ubuntu Playground**:

https://killercoda.com/playgrounds/scenario/ubuntu

> ***Web tool:*** *Rehearse the attack/defence scenarios in this lab with the free browser* ***Cybersecurity Simulator*** *— https://alfredang.github.io/cybersecuritysimulator/.*

Step 1 — Install tools

```
apt update && apt install -y openssl curl docker.io git auditd
systemctl start docker
systemctl start auditd
```

Step 2 — Deprecated cipher / TLS version

```
docker run -d --name oldssl -p 8443:443 \
  -v /etc/ssl/certs:/certs:ro \
  alpine sh -c "apk add --quiet openssl && openssl req -x509 -newkey rsa:2048
-nodes -keyout /tmp/k -out /tmp/c -subj '/CN=test' -days 1 && openssl s_server
-accept 443 -cert /tmp/c -key /tmp/k -tls1 -www" 2>/dev/null

sleep 4

# Try modern client (TLS 1.0 disabled by default)
echo | openssl s_client -connect localhost:8443 -tls1_3 2>&1 | grep -E
'(version|alert|verify return)' | head -3

# Negotiate weak version
echo | openssl s_client -connect localhost:8443 -tls1 2>&1 | grep 'Protocol' |
head -1

docker rm -f oldssl
```

Fix: disable SSLv3, TLS 1.0, TLS 1.1; require TLS 1.2+ with strong AEAD ciphers (AES-GCM, ChaCha20-Poly1305).

Step 3 — Test a public site's cipher suite

```
nmap --script ssl-enum-ciphers -p 443 example.com 2>/dev/null | head -40 || \
  echo | openssl s_client -connect example.com:443 -servername example.com
2>/dev/null | grep -E '(Protocol|Cipher)'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step 4 — Privilege escalation detection

```
auditctl -w /etc/sudoers -p wa -k sudo-changes
echo '# test' >> /etc/sudoers
ausearch -k sudo-changes 2>/dev/null | tail -5

# Find SUID binaries (potential privesc)
find / -perm -4000 -type f 2>/dev/null | head -10
```

Fix: restrict sudoers, audit SUID, run `linpeas`/`Lynis` regularly.

Step 5 — Unauthorized access / brute force

Tail SSH log for failed auth:

```
journalctl -u ssh --since "1 hour ago" 2>/dev/null | grep -i "failed\|invalid"
| head -5
```

Mitigation: install `fail2ban`:

```
apt install -y fail2ban
systemctl start fail2ban
fail2ban-client status sshd 2>/dev/null
```

Step 6 — Leaked credentials in source control

Ready-made file: `config.py` — you can download it instead of typing this block.

```
mkdir -p /tmp/leak && cd /tmp/leak && git init -q
cat > config.py <<'EOF'
AWS_ACCESS_KEY="AKIAIOSFODNN7EXAMPLE"
AWS_SECRET="wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
EOF
git add . && git -c user.email=x@x -c user.name=x commit -q -m "oops"

# Detect with gitleaks
docker run --rm -v /tmp/leak:/repo zricethezav/gitleaks:latest detect -s /repo
--no-git -v 2>&1 | head -20
```

Fix: rotate the leaked key immediately; add a pre-commit `gitleaks` hook; store in Vault (Lab 26).

Step 7 — Software vulnerability + unauthorized software

```
# Vulnerable package check (Trivy from Lab 22)
docker run --rm -v /tmp:/scan aquasec/trivy fs --severity CRITICAL /scan 2>&1
| tail -10 || true

# Unauthorized software inventory: list packages added in last 7 days
grep " install " /var/log/dpkg.log 2>/dev/null | tail -10
```

Fix: allow-list packages; block unsigned repos; scan in CI (Lab 30).

Step 8 — TLS handshake debugging cheatsheet

Symptom	Likely cause
`unable to get local issuer certificate`	Missing CA bundle
`handshake failure`	Cipher mismatch

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

`certificate has expired`	Renew cert (Let's Encrypt)
`hostname mismatch`	SAN not matching SNI
`ssl3_get_record:wrong version number`	Plain HTTP on TLS port

Step 9 — Cleanup

```
sed -i '/# test/d' /etc/sudoers
rm -rf /tmp/leak
```

Test it

Run these checks to prove the lab worked before you move on:

```
echo | openssl s_client -connect localhost:8443 -tls1_3 2>&1 | grep -E
'(alert|Protocol)' | head -3
echo | openssl s_client -connect localhost:8443 -tls1 2>&1 | grep 'Protocol' |
head -1
ausearch -k sudo-changes 2>/dev/null | tail -5
find / -perm -4000 -type f 2>/dev/null | head -10
docker run --rm -v /tmp/leak:/repo zricethezav/gitleaks:latest detect -s /repo
--no-git -v 2>&1 | head -20
```

Expected: Run the TLS checks while the `oldssl` container from Step 2 is still running. The TLS 1.3 client is **refused** (handshake failure / alert) while the `-tls1` client negotiates `Protocol : TLSv1` — proving the server only offers a deprecated version; `ausearch` shows an audit record for the write to `/etc/sudoers` under key `sudo-changes`; the SUID list includes the usual `/usr/bin/sudo`, `/usr/bin/passwd` and friends; and gitleaks reports **2 leaks** in `config.py`, naming the AWS access key and secret it found.

What you learned

- Negotiate TLS versions and ciphers from the CLI.
- Detect privesc, brute force, and leaked secrets.
- Common TLS errors and their root causes.

Free tools used

- OpenSSL — https://www.openssl.org
- testssl.sh — https://testssl.sh
- SSL Labs — https://www.ssllabs.com/ssltest
- gitleaks — https://github.com/gitleaks/gitleaks
- TruffleHog — https://github.com/trufflesecurity/trufflehog
- fail2ban — https://www.fail2ban.org
- auditd (built-in)

Files in this lab

File	Purpose
`config.py`	Step 6 file with dummy AWS example credentials that gitleaks detects.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 36 — Troubleshoot Container & Resource Limits

Goal: Combine the Lab 33–35 methods to triage real container failures with a repeatable state → logs → stats → mounts → network workflow.

Figure 36 — the workflow you build in this lab.

Runs on: Docker Desktop — https://www.docker.com/products/docker-desktop/

Tools used: docker (inspect/stats/logs), a triage.sh script, k9s/ctop

In this final lab you will combine the methods from Labs 33-35 to triage real container failures: **CrashLoopBackOff**, **OOMKilled**, image pull failures, hostname/DNS resolution inside containers, and read-only filesystem errors.

Lab platform

Run this lab either way:

- **Killercoda Ubuntu Playground** (nothing to install) — https://killercoda.com/playgrounds/scenario/ubuntu
- **Docker Desktop** on your own machine — https://www.docker.com/products/docker-desktop/

> *On Docker Desktop drop the `apt install docker.io` and `systemctl start docker` steps — the engine is already running. Everything else is identical.*

Step 1 — Setup

```
apt update && apt install -y docker.io curl
systemctl start docker
```

Step 2 — CrashLoopBackOff (process exits immediately)

```
docker run -d --name crash --restart=on-failure alpine sh -c "exit 1"
sleep 3
docker ps -a --filter name=crash
docker logs crash
```

Kubernetes equivalent:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "kubectl describe pod <name>  # look at 'Last State: Terminated, Reason'"
```

Fix: read logs, fix the entrypoint command, ensure the binary exists and exits 0.

Step 3 — OOMKilled

```
docker run -d --name oom --memory=20m --restart=no alpine \
  sh -c "yes hello | head -c 100m"
sleep 3
docker inspect oom --format='{{.State.OOMKilled}} {{.State.ExitCode}}
{{.State.Error}}'
docker rm -f oom
```

`OOMKilled=true` is the **smoking gun**. Fix: raise the limit (`--memory=128m`) or fix the leak.

Step 4 — Image pull failure

```
docker run -d --name pull docker.io/myorg/no-such-image:1.0 2>&1 | tail -3
```

Fix: check image name, registry auth (`docker login`), private-registry credentials in K8s `imagePullSecrets`.

Step 5 — DNS inside containers

```
docker run --rm alpine nslookup kubernetes.default 2>&1 | head -3
docker run --rm --dns 1.1.1.1 alpine nslookup example.com | tail -3
```

In K8s:

```
echo "kubectl exec -it <pod> -- nslookup kubernetes.default"
echo "If this fails -> CoreDNS down or NetworkPolicy blocking 53/udp."
```

Step 6 — Read-only filesystem errors

```
docker run --rm --read-only alpine sh -c "echo x > /tmp/y" 2>&1 | tail -1
docker run --rm --read-only --tmpfs /tmp alpine sh -c "echo x > /tmp/y &&
cat /tmp/y"
```

Read-only roots are **good security** — give writable paths via `tmpfs` or volumes.

Step 7 — Resource quota exhaustion (sizing)

```
docker run -d --name big --memory=2g --cpus=4 alpine sleep 1000 2>&1 || \
  echo "If host has insufficient resources -> pending / failed scheduling"
docker rm -f big 2>/dev/null
```

K8s analogue: `kubectl describe pod` → `FailedScheduling: 0/1 nodes available: insufficient memory`.

Step 8 — End-to-end triage script

Ready-made file: `triage.sh` — you can download it instead of typing this block.

```
cat > /tmp/triage.sh <<'EOF'
#!/bin/bash
NAME=$1
echo "=== State ==="; docker inspect $NAME --format='{{.State.Status}}
OOM={{.State.OOMKilled}} Exit={{.State.ExitCode}}'
echo "=== Logs (last 30) ==="; docker logs --tail 30 $NAME 2>&1
echo "=== Stats ==="; docker stats --no-stream $NAME 2>/dev/null
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "=== Mounts ==="; docker inspect $NAME --format='{{range .Mounts}}
{{.Source}} -> {{.Destination}} ({{.Mode}}){{println}}{{end}}'
echo "=== Network ==="; docker inspect $NAME --format='{{range $k,
$v := .NetworkSettings.Networks}}{{$k}}={{$v.IPAddress}}{{end}}'
EOF
chmod +x /tmp/triage.sh

docker run -d --name demo nginx:alpine
/tmp/triage.sh demo
docker rm -f demo
```

Step 9 — Master decision flow

1. **Pod won't start** → image pull, command, secrets.
2. **Pod crashloops** → application logs, exit code.
3. **Pod OOM** → resize memory, fix leak.
4. **Pod CPU throttled** → resize CPU, optimise code.
5. **Pod can't reach service** → DNS, NetworkPolicy, ServiceAccount RBAC.
6. **Pod read-only error** → mount tmpfs/volume.

Step 10 — Closing summary

You have now hit every CV0-004 troubleshooting category:

Category	Lab
Deployment	33
Network	34
Security	35
Container	36

Test it

Run these checks to prove the lab worked before you move on:

```
docker ps -a --filter name=crash --format '{{.Names}}\t{{.Status}}'
docker inspect oom --format='{{.State.OOMKilled}} {{.State.ExitCode}}'
docker run --rm --read-only --tmpfs /tmp alpine sh -c "echo x > /tmp/y &&
cat /tmp/y"
docker run --rm --dns 1.1.1.1 alpine nslookup example.com | tail -3
/tmp/triage.sh demo
```

Expected: Run each check while that step's container still exists. The `crash` container cycles through `Restarting (1)` / `Exited (1)` — the Docker equivalent of CrashLoopBackOff; `docker inspect oom` prints **`true 137`**, the OOMKilled smoking gun; the read-only container with a tmpfs mount succeeds and prints `x` where the plain `--read-only` run failed with *Read-only file system*; the DNS lookup resolves `example.com`; and `triage.sh demo` prints the State / Logs / Stats / Mounts / Network sections for the running nginx container.

What you learned

- Read state, logs, stats, mounts, network — in that order.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- OOM, CrashLoop, ImagePull, DNS, RO-fs are the daily five.
- A short triage script saves 10 minutes per incident.

Free tools used

- Docker — https://www.docker.com
- kubectl + k9s — https://k9scli.io
- ctop (container top) — https://github.com/bcicen/ctop
- lazydocker — https://github.com/jesseduffield/lazydocker

Files in this lab

File	Purpose
`triage.sh`	Step 8 end-to-end container triage script (state, logs, stats, mounts, network).

Quick Command Reference

Task	Command
Install Docker on the Killercoda VM	`apt update && apt install -y docker.io`
Run a detached container (IaaS sim)	`docker run -dit --name x ubuntu:22.04 bash`
Inspect running containers when triaging	`docker ps / docker logs / docker inspect`
Read live CPU/memory (spot OOM & oversubscription)	`docker stats --no-stream`
Smoke-test a service endpoint	`curl -s http://localhost:PORT
Inspect VPC subnets, routes and security groups	`ip netns / ip route / iptables -L`
Provision Infrastructure as Code	`terraform init / plan / apply`
Apply Configuration as Code idempotently	`ansible-playbook -i inventory.ini play.yml`
Operate a Kubernetes (k3s) workload	`kubectl get/scale/rollout`
Scan for vulnerabilities and misconfiguration	`trivy image / fs / config`
Core source-control workflow	`git commit / push / checkout -b / merge`

Support & Assessment

- **Trainer / enquiries:** enquiry@tertiaryinfotech.com · +65 6100 0613
- **Courseware & assessment (LMS):** https://lms-tms.tertiaryinfotech.com/
- **Website:** www.tertiarycourses.com.sg

Assessment Flow

At the assessment step, follow this order:

1. **TRAQOM** — scan the TRAQOM QR code on the LMS and complete the survey.
2. **Assessment Digital Attendance**.
3. **Assessment** — Written Assessment (WA) + Practical Performance (PP).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. **Submit** your assessment answers on the LMS.
5. **Sign** the Assessment Summary Record.

Courseware and the assessment are on the LMS (https://lms-tms.tertiaryinfotech.com/). For a two-day class the 2-hour assessment runs 4:30–6:30 PM on the final day (1 hr WA + 1 hr PP).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg